

C64 Visual Assembler — Felhasználói kézikönyv

Verzió: 2.4.1

Vizuális, blokk-alapú 6502 assembler a Commodore 64-hez. A programot utasításblokkok fogd-és-vidd módszerrel történő elrendezésével építed fel, a generált assembly és gépi kód pedig valós időben frissül.

Ez a magyar fordítás az angol nyelvű **Visual Assembler Manual.md** alapján készült; hivatalos forrásnak az angol változat számít.

Tartalomjegyzék

- [1. A felület áttekintése](#)
- [2. Blokk paletta](#)
- [3. Programterület](#)
- [4. ASM nézet](#)
- [5. Beállítások és eszköztár](#)
- [UltimateBasic mód](#)
- [6. Expert mód](#)
- [7. Címzési módok](#)
- [8. Szabványos 6502 utasítások](#)
- [9. Makró blokkok — referencia](#)
- [10. Debugger integráció](#)
- [11. Tudásbázis linkek](#)
- [12. D64 export és futtatás](#)
- [12b. CRT export \(Magic Desk 64K cartridge\)](#)
- [13. Hardver beállítások](#)
- [14. Vizuális szerkesztők \(Toolkit\)](#)

A 2.4.1 verzió újdonságai

- **Lengyel és olasz felületi nyelv** — teljes felület-fordítás (menük, dialógusok, mnemonik leírások, Ultimate Basic parancsreferencia) a meglévő magyar, angol, spanyol, német és holland mellett.
- **Online Help eszköztár-gomb** — új ikon-gomb a Debug után, a többnyelvű dokumentációs oldalt (c64va.tech/docs.html) nyitja meg közvetlenül az appból.
- **Mnemonik leírások központosítása:** a nyelvenkénti mnemonik leírások a közös fordítási fájlba kerültek a többi felületi szöveg mellé. Emellett javítva lett egy régóta fennálló hiba, ahol a `PRINT`, `PRINT_HEX`, `CLEAR_SCREEN`, `WAIT_KEY`, `SET_BORDER`, `SET_BG`, `IRQ_SETUP` és `RAND` angol leírásait egy véletlenül ottmaradt spanyol szöveg felülírta.

A 2.4.0 verzió újdonságai

- **D64 Editor** — teljes értékű lemezkép-böngésző az eszköztáron (a Curve Editor mögött). Meglévő `.d64` megnyitása, új üres lemez létrehozása, vagy a nyitott lemez közvetlen indítása VICE-ben — mindezt egy, a többi vizuális szerkesztőhöz hasonló `Files ▾` menüből. Lásd: [D64 Editor \(meglévő lemezkép böngészése és szerkesztése\)](#).
 - **Program hozzáadás / kimentés / átnevezés / törlés a D64 Editorban** — helyi fájl hozzáadása a lemez directory-jához, kijelölt bejegyzés kimentése `.prg` -ként, bejegyzés átnevezése helyben a táblázatban, vagy törlése — minden művelet közvetlenül a `.d64` fájlban hajtodik végre `c1541` -en keresztül, külön mentés lépés nélkül.
 - **Betöltési cím, kicsomagolási cím és Exomizer a D64 Editorban** — fejléc nélküli nyers fájl hozzáadásakor megadható egy opcionális betöltési cím, egy Exomizer kicsomagolási célcím, és a fájl tömöríthető hozzáadás közben, ugyanazokkal a `mem / sfx` tömörítési módokkal, mint a Build D64 dialógus extra fájljainál. Egy már saját fejléccel rendelkező `.prg` -nél ezek a mezők teljesen kimaradnak.
 - **Lemez bejegyzés típus választó** — az újonnan hozzáadott fájlhoz PRG / SEQ / USR / REL típus választható, nem csak PRG.
 - **Hiteles directory lista** — a D64 Editor fájllistája a becsomagolt C64 Pro fonttal, nagybetűsen jelenik meg, a klasszikus `LOAD"$",8` hatásért.
 - **Javítva:** a D64 Editorban egy bejegyzés átnevezésekor a szövegmezőbe kattintás többé nem törli azonnal a szerkesztést.
 - **Javítva:** világos témában a mód-jelző eszköztár-jelvény (BLOCK MODE / EXPERT MODE / ...) sötétebb és olvashatóbb, a csillanás animáció ismét látszik.
-

A 2.3.9 verzió újdonságai

Öt assembler kényelmi funkció, mind használható Expert mód szövegében és (ahol értelmes) blokként is. Mindegyikhez tartozik egy részletes szakasz lentebb:

- *** bármely kifejezésben** — a program counter szimbólum mostantól működik operandus-kifejezéseken belül is, nem csak önmagában: `BNE *-5`, `JMP *+20`, `LDA #<*`, `LDA #>(*+63)`. Az érték után álló `*` (`STRIDE*2`) továbbra is szorzás. Lásd a [7. szakaszt → A * program counter kifejezésekben](#).
 - **Lokális (pontos) címkék** — a `.loop` -hoz hasonló címke a legközelebbi előtte lévő *globális* (nem pontos) címke hatókörébe tartozik, így a `DrawSprite` és a `ClearScreen` is definiálhatja a saját `.loop` -ját ütközés nélkül. Lásd: [Lokális \(pontos\) címkék](#).
 - **Long-branch pszeudo-utasítások** — az `LBNE`, `LBEQ`, `LBCC`, `LBCS`, `LBMI`, `LBPL`, `LBVC`, `LBVS` egy `JMP` fölötti invertált elágazásra fordul (mindig 5 byte), így a célpont tetszőleges távolságra lehet. Új **Hosszú ugrások** paletta-kategória. Lásd: [LBNE / LBEQ / ... \(Hosszú ugrások\)](#).
 - **.assert direktíva** — a `.assert end - start <= 256` vagy `.assert * < $A000`, "üzenet" fordítási időben kiértékelődik, és ha a kifejezés hamis, a build a tényleges értékkel együtt hibára fut. Lásd: [.ASSERT](#).
 - **Önmódosító kód (SMC) operandus-címkék** — a `LDA value:#$00` a `value` címkét az utasítás operandus-byte-jára helyezi, így a `STA value` közvetlenül azt módosítja. Lásd: [Önmódosító kód operandus-címkék](#).
 - **Barátságosabb tartományon kívüli branch-hibák** — a `-128...+127` tartományon kívülre eső elágazás mostantól pontosan jelzi, mennyivel lóg túl, és javasolja a megfelelő `LBxx` hosszú ugrást.
-

1. A felület áttekintése

Az app három fő panelre oszlik:

Panel	Leírás
Bal — Paletta	Minden elérhető utasítás- és makró-blokk. Keresés vagy böngészés kategória szerint.
Közép — Program	A programod. Ide húzod a blokkokat, itt rendezed át őket, itt szerkeszted az operandusokat.
Jobb — Kimenet	Élő ASM nézet és/vagy memória monitor kimenet.

A fejléc jobb szélén lévő mód-jelvény mutatja az aktív **Blokk**, **Expert** vagy **Ultimate Basic** szerkesztőt. Azonnal frissül, amikor a szerkesztőmód változik.

2. Blokk paletta

A bal oldali paletta minden elérhető blokkot kategóriákba csoportosítva listáz:

- **Data movement** — LDA, LDX, STA, STX, ...
- **Arithmetic** — ADC, SBC, INC, DEC, CMP, ...
- **Logic** — AND, OR, EOR, BIT
- **Jumps & Branches** — JMP, JSR, RTS, BNE, BEQ, ...
- **Long branches** — LBNE, LBEQ, LBCC, LBCCS, LBMI, LBPL, LBVC, LBVS (tetszőleges távolságra ugrik; lásd a 8. szakaszt)
- **Register operations** — TAX, TAY, INX, DEX, ...
- **Shift & Rotate** — ASL, LSR, ROL, ROR
- **Stack** — PHA, PHP, PLA, PLP
- **System** — CLC, SEC, NOP, BRK, ...
- **Illegal instructions** — LAX, SAX, DCP, ...
- **Structure** — LABEL, COMMENT, REGION, ENDREGION
- **Macros** — LOOP, NEXT, FOR, ENDF, PUSH, PULL, END, TEXT, BYTE, WORD, FILL, ALIGN, ASSERT, STRING, DATA, RAWBYTES, RAWTEXT, PETSCII, CHARSET, INCBIN, SID, INCLUDE, TABLE, ORG, MACRO, ENDM, INVOKE, IF, ELSE, ENDIF, VAR, WHILE, ENDW, REPEAT, UNTIL, MEMCPY, MEMSET, PRINT, PRINT_CHAR, PRINT_HEX, CLEAR_SCREEN, WAIT_KEY, DELAY, SET_BORDER, SET_BG, IRQ_SETUP, RAND, SPRITE_INIT, SPRITE_POS, WAIT_RASTER, JOYSTICK, MOUSE, SPRITE_COL, LOADFILE, REU_CHECK, REU_STASH, REU_FETCH, REU_SWAP, TURBO_SET, SUPERCPU_DETECT, TURBO_ENABLE, MAP_COPY, MAP_COPY16X16, SPRITE_ANIM, SCORE_BCD

A paletta tetején lévő **keresőmezővel** név szerint szűrhetesz. Kattints az **Add selected block** gombra, vagy húzz egy blokkot a programterületre.

3. Programterület

- **Fogd-és-vidd** blokkokat a palettáról, vagy **rendezd át** a meglévő blokkokat a fogantyújuk (≡) húzásával.
- Minden blokk mutatja a **mnemonikját**, az **operandus-mezőjét** és a **címzési mód választót** (ahol értelmezhető).
- A ▸ / ▾ kapcsolóval csukod be vagy nyitod ki a blokkot.
- A blokk × **(törlés)** gombjával távolítod el.
- A **Collapse All** gomb egyszerre csukja be az összes blokkot.

Blokk panel minimap

A Program panel fejlécében van egy kapcsolható **minimap** gomb. Bekapcsolva egy keskeny 56 px -es canvas-sáv jelenik meg a panel jobb szélén, amely minden blokkot színkódolt vízszintes sávként ábrázol:

Sáv színe	Blokk típusa
Cián	Címkék
Kék/lila	Makrók és direktívák
Sárga	Utasítások
Zöld	Megjegyzések és üres sorok
Piros	Érvényesítési hibás blokkok

A becsukott blokkok csökkentett átlátszatlansággal jelennek meg. Kattints vagy húzz bárhol a minimapon, hogy a programlistát arra a pozícióra görgesd. A viewport-jelző (kiemelt színű téglalap) a lista látható részét követi. Az állapot a UI beállításokban tárolódik (`blockMinimap` kulcs).

Operandus bevitel

- Elágazás/ugrás utasításoknál (`BNE` , `JMP` , `JSR` , stb.) egy **címke-választó** legördülő jelenik meg — kattints egy definiált címkére a beszúráshoz.
- A számformátum követi az eszköztár **HEX / DEC** kapcsolóját (lásd az 5. szakaszt).

4. ASM nézet

A jobb oldali panel valós időben mutatja a generált kimenetet.

Kimeneti módok

Mód	Leírás
ASM	6502 assembly forrás címekkel és címkékkel
Monitor	Hex / byte dump (C64 monitor stílus)
Disasm	Tiszta 6502 disassembly: cím · hex byte-ok · mnemonikok feloldott numerikus operandusokkal. A makrók egyedi utasításokra bomlanak (TEXT → LDA/STA párok, LOOP → LDX, stb.). A BYTE/WORD/FILL adat darabolt hex dumpként jelenik meg. A kimenetben nincsenek makró-nevek, megjegyzések vagy jegyzetek.
Both	Fent ASM, lent monitor
Disassembler	Ugyanaz, mint a Disasm — külön fül a disassembly nézethez
Toolkit	C64 referencia panel: 16 színes paletta-minta + PETSCII vezérlőkód és nyomtatható-karakter puska. Csak olvasható — részletekért lásd lentebb a „Toolkit fül” alszakaszt.
Options	Program beállítások panel — számformátum, makró-forrás kapcsoló, debugger paraméterek

Toolkit fül

Az ASM nézet **Toolkit** füle egy csak olvasható gyorsreferencia panel — soha nem módosítja a programodat. Két szakasz:

Szakasz	Tartalom
C64 színpaletta	16 mintás rács, minden C64 színnel, indexével (0–15 / \$00 – \$0F) és nevével. Kattints egy mintára a hex indexének vágólapra másolásához. Vidd fölé az egeret a szín nevéért (Light Blue, Brown, stb.).
PETSCII vezérlőkódok	Gyakori vezérlőkódok a CHROUT (\$FFD2) számára: színváltó kódok (\$05 fehér, \$1C piros, \$1E zöld, \$1F kék, ...), kurzormozgatás (\$11 / \$1D / \$91 / \$9D), inverz be/ki (\$12 / \$92), \$93 képernyőtörlés, \$8E / \$0E karakterkészlet-váltás. Emellett nyomtatható tartomány puska (32-64 írásjelek, 65-90 A-Z, 91-95 zárójelek, 96-127 grafika, 160-191 shiftelt grafika, 192-223 tükör).

A Toolkit a leggyorsabb módja egy színindex vagy egy PETSCII vezérlő byte megkeresésének anélkül, hogy elhagynád a szerkesztőt.

Options fül

Az **Options** fül tartalmazza a kódgenerálást és a kimenet megjelenítését befolyásoló beállításokat:

- **Macro source** — bekapcsolva a makró-definíciós blokkok (MACRO...ENDM) a forráskódjukat inline mutatják az ASM nézetben.
- **Program kezdőcím** — mostantól a programterületen lévő **ORG blokkal** állítható be, nem külön beviteli mezővel. Az első ORG blokk határozza meg a program betöltési címét; a további ORG blokkok újabb szegmenseket indítanak eltérő címeken.
- **Debugger params** — három inline kapcsoló, amely szabályozza, mely flag-ek kerülnek átadásra a külső debuggernek indításkor:
 - **-jmp BE/KI** — betöltés után közvetlenül a program kezdőcímére ugrik.
 - **-unpause BE/KI** — betöltéskor azonnal folytatja a debuggert.
 - **-wait ms BE/KI** — **-wait <ms>** késleltetést ad a folytatás előtt; válassz 500 ms-t vagy 1000 ms-t a legördülőből.
- **Compile info** — összefoglalót mutat a lefordított programról (kód kezdőcíme, méret, BASIC SYS stub állapota).

ASM sorra kattintás

Kattints bármelyik sorra az ASM nézetben, hogy **kiemeld a hozzá tartozó blokkot** a programterületen.

ASM sorszámok

Az ASM panel **sorszámokat** jelenít meg (001 | , 002 | , ...), hogy megkönnyítse a hibakeresést, amikor egy fordítási hiba egy konkrét sorra mutat.

- A vizuális sorszámok csak diagnosztikai célúak.
- A **Copy ASM** továbbra is a tiszta forrásszöveget másolja, sorszám-prefixek **nélkül**.

Fordítási folyamat modal

Nehezebb műveletek alatt egy középre igazított folyamatjelző modal jelenik meg progress bárral:

- **Run in VICE** — PRG fordítása/buildelése és az emulátor indítása.
- **Debug** — PRG fordítása/buildelése és a debugger indítása.
- **Load .asm file** — **.asm** fájl megnyitása Expert módban, és blokkok materializálása a forrásból.

A modal automatikusan bezárul, amikor a művelet befejeződik vagy meghiúsul.

5. Beállítások és eszköztár

Vezérlő	Leírás
Számrendszer (HEX / DEC / BIN)	Beállítja az operandusok megjelenítési/beviteli formátumát az egész felületen. BIN módban az értékek binárisan, % prefixszel jelennek meg (pl. %11111000). Az ASM nézet mindig a saját formátumában mutatja az egyes blokkokat.
Nyelv	A felület nyelvének váltása angol, magyar, spanyol, német és holland (Nederlands) között
Téma	Light / Dark / OLED / Commodore 77 — a Settings menü témaválasztójából. Az OLED tiszta fekete hátteret használ AMOLED kijelzőkhöz. A Commodore 77 egy neon-sárga-feketén téma; ha ez az aktív téma, az indítási splash panel a téma panel-színét használja (a message card-hoz igazítva), egy kisebb, dedikált Commodore 77 logót és egy sárga progress bárt mutat. A választott téma a következő indításkor az első kirajzolás előtt érvényesül.
CRT retro mód	Teljes képernyős CRT szűrőt kapcsol: scanline-ok, foszfor vignettálás, villódzás és hordódistorzió. Az állapot munkamenetek között mentődik.
Show memory panel	Globális kapcsoló, amely megjeleníti vagy elrejtí a teljes C64 memória panelt
BASIC SYS stub	Egy BASIC sort tesz a program elé, amely SYS-szel hívja a program originjét
Sample	Beépített mintaprogram betöltése
Zoom in / out	A blokk-felület méretezése (minden blokk-elemre hat)
Save Project	Az aktuális program mentése .json projektfájlként
Save Program As	Az aktuális program mentése .json projektfájlként, minden alkalommal új fájl-dialógussal
Load Project	Korábban mentett projekt betöltése
Save Workspace	A jelenleg nyitott, fájlhoz kötött fülek pontos halmazának mentése — az aktív füllel és minden fül szerkesztőmódjával (Block/Expert/Ultimate Basic) — .vaws munkaterület-fájlba
Save Workspace As	Az aktuális munkaterület mentése, minden alkalommal új fájl-dialógussal
Open Workspace	Minden nyitott fül bezárása és a .vaws munkaterület-fájlban tárolt fájlkészlet újrainvitása
Set working folder	A fájlválasztók és mentés-dialógusok alapértelmezett mappájának kiválasztása. Az útvonal az app konfigurációjában tárolódik, és a menü-előnézetek az útvonal végét tartják láthatóan.
Open Project (Menu → File)	Több-fájlos .proj projekt megnyitása, minden forrásfájl fülként megnyitva
Save Project (Menu → File)	Az aktuális .proj projekt mentése (a projekt panelnek nyitva kell lennie)
Close Project (Menu → File)	A jelenleg nyitott projekt és minden fájlfülének bezárása. Rákérdez a nem mentett változtatások mentésére. A projekt panel visszaáll üres állapotba.
Load .asm file	.asm fájl megnyitása Expert módban, és szöveges 6502 ASM importja az aktuális fülbe
Save PRG	A lefordított bináris exportja .prg fájlként
Build CRT	A program exportja 64K Magic Desk (.crt , cartridge type 19) fájlként. Lásd a 12b. szakaszt .

Vezérlő	Leírás
Run (osztott gomb)	A fő ► Run gomb az aktuális módot futtatja; a ▼ nyíllal válthatsz: Run as PRG (fordítás és VICE közvetlen indítása), Run via D64 (.d64 lemezképbe csomagolás és VICE indítása), vagy Run on hardware (PRG küldése egy C64 Ultimate / 1541 Ultimate eszközre). Lásd a 12. és 13. szakaszt .
Debug (RetroDebugger)	Fordítás és indítás RetroDebuggerben breakpointokkal, szimbólumokkal és autostart flag-ekkel (lásd a 10. szakaszt)
Run with Exomizer	Jelölőnégyzet a Settings menüben — bekapcsolva minden Run és Build művelet <code>exomizer sfx sys -</code> szel tömöríti a PRG-t indítás vagy mentés előtt. Működik a Run as PRG, Run via D64, Run on Hardware, Build PRG és Build D64 műveletekkel. Először állítsd be az Exomizer futtathatót a Hardware Settings -ben.
Automatikus snapshot mentés	Jelölőnégyzet a Hardware Settings → Snapshot alatt. Bekapcsolva az app automatikusan készít snapshotot kb. 2,5 másodperccel azután, hogy abbahagyta egy fül szerkesztését. Kapcsold ki, ha csak kézi snapshot mentést szeretnél.
Hardware Settings	Megnyitja a hardver konfigurációs dialógust — VICE, Exomizer, RetroDebugger és C64 Ultimate beállítása (host, jelszó, kapcsolat-teszt). Lásd a 13. szakaszt .
New program...	Megerősítő dialógust nyit, majd törli az összes blokkot a programterületről
Collapse All	Minden blokk becsukása
About	Verzióinformáció
What's New	Changelog
Knowledge Base	Referencia linkek (6502 opcode-ok, C64 KERNAL, memóriatérkép, színek)
Check for Update	Az itch.io oldal megnyitása egy újabb kiadás ellenőrzéséhez

Projekt snapshotok

A projekt snapshotok lemezen tárolt sidecar JSON fájlakként vannak elmentve, nem a localStorage-ban. Az aktuális projektfájlhoz kötődnek, ha van ilyen, így az előzmény túléli az újraindításokat és követi a projektet.

- A **Menu** → **Build** → **Save snapshot** megnyitja a snapshot dialógust, és elmenti az aktuális blokk-állapotot plusz az Expert ASM szöveget.
- A **Menu** → **Build** → **Restore previous version** közvetlenül visszaállítja a legutóbbi snapshotot.
- A **Menu** → **Build** → **Snapshot history** megnyitja a dialógust, ahol jegyzeteket adhatsz, régebbi bejegyzéseket állíthatsz vissza vagy törölhetsz.
- A **Hardware Settings** → **Snapshot** → **Automatikus snapshot mentés** szabályozza, hogy az app automatikusan készít-e snapshotokat szerkesztések után. Az alapértelmezett késleltetés kb. 2,5 másodperc, és a beállítás fülönként érvényes.
- Ha egy projektet még nem mentettél el, a snapshotok az app konfigurációs könyvtárban tárolódnak, amíg a projekt fájlvonalat nem kap.

Munkaterületek

Egy **munkaterület** (`.vaws` fájl) megjegyzi, mely valós, lemezen lévő fájlok voltak nyitva az egyes füleken — beleértve minden fül szerkesztőmódját és azt, hogy melyik fül volt aktív —, így később újranyithatod pontosan ezt a készletet. Ez elkülönül a `.proj` projekttől: egy munkaterület átfoghatja a Block/Expert `.json` projektfájlok, önálló `.asm` fájlok és Ultimate Basic `.ub` / `.proj` források tetszőleges keverékét több fülön keresztül.

- A munkaterületek **automatikusan mentődnek** néhány száz milliszekundummal egy változtatás után, amint már mentetted vagy megnyitottad egyet.
- Az app induláskor **automatikusan visszaállítja az utolsó munkaterületet**, így a nyitott füleid ott folytatódnak, ahol abbahagytad.
- Csak a lemezen lévő valós fájlhoz kötött fülek mentődnek a munkaterületbe; egy nem mentett mintát vagy csak memóriában lévő programot tartalmazó fülnek nincs mit megőriznie, így az kihagyódik (értesítéssel, ha egyik nyitott fül sem alkalmas).
- Egy munkaterület megnyitása először bezár minden jelenleg nyitott fület — a folytatás előtt megerősítést kér.
- Ha egy munkaterület olyan fájlra hivatkozik, amelyet azóta áthelyeztek vagy töröltek, az a bejegyzés kihagyódik, és betöltés után név szerint jelentődik.

.asm fájl betöltése (gyorsreferencia)

Az Expert-mód `.asm` betöltő elfogadja a gyakori 6502 forrásmintákat, és blokkokká alakítja őket:

- `* = $1500` → ORG blokk
- `Label:` → LABEL blokk
- `Label: .byte 0` → LABEL + BYTE blokkok
- `.byte ...` → BYTE blokk
- `; comment` (vagy inline `; ...`) → COMMENT blokk
- utasítások (`lda`, `jsr`, `beq`, `stb`.) → utasításblokkok felismert címezési móddal

Import parse-jegyzetek és jó gyakorlatok

- A lokális címkék, mint a `.wait`, szabványos címkeként importálódnak (a pont eltávolítva), és a hivatkozások ennek megfelelően normalizálódnak.
- A `($zp),Y / ($zp,X)` stílusú címezéshez használj konkrét zero-page byte-ot (`$FB`, `$FC`, stb.) a legjobb kompatibilitásért.
- Kerüld az elágazási kontextusban a hexnek tűnő, kétértelmű rövid címkéket (`cc1`, `dead`, `beef`); inkább `loop_cc1`-hez hasonló neveket használj.
- Ha a programod adattal (`.byte`) kezdődik a futtatható kód előtt, tegyél egy explicit belépési ugrást (például `JMP Start`) a tetejére.

ASM import (Kick Assembler)

A Program menü **ASM import** gombja nyers Kick Assembler forrást olvas be egy új blokk-módú fülbe. Ez elkülönül a fenti Expert-mód `Load .asm file`-től — az egyéni tooltípe jelzi, hogy **csak Kick Assembler kód támogatott** (más assemblerek részben parse-olhatók, de a round-trip nem garantált).

Támogatott minták:

- `.pc = $XXXX` origin direktíva → ORG blokk
- `.const NAME = value`, `.label NAME = value` → CONST equate
- `.macro NAME(p1, p2, ...) { ... } { / }` kapcsos test-tel vagy `.endm`-mel → felhasználói makró-definíció
- Makró-hívás `NAME(args)`, Kick kettőspont-prefix `:NAME(args)` és `.invoke NAME(args)` — mind a Kick kettőspontos formán keresztül round-trip-el
- `@local` címkék (`@loop:`, `BEQ @loop`) megőrzik a `@` prefixet változatlanul
- Operandus `label + N / label - N` (pl. `STA mod1+2`, `LDA xp+1`)
- Sor-megjegyzések `// ...` és `;` — mindkettő elfogadott, a `/* ... */` blokkok egyetlen megjegyzés-sorként kezelődnek

- BASIC autostart átvitel: ha a program `$0801` -en kezdődik a szabványos `SYS 2061` byte stubbal (`.byte $0B,$08,$0A,$00,$9E,$32,$30,$36,$31,$00,$00,$00`), a fordító szó szerint bocsátja ki a PRG-t, ahelyett hogy egy második BASIC SYS-t csomagolna köré

Ismert korlát:

- Azok a konstansok, amelyek zero-page címre oldódnak fel (például a `.const BYTEADDR = $FC STA BYTEADDR` -ként használva) jelenleg abszolút módú utasításokra fordulnak (3 byte) a zero-page (2 byte) helyett. A lefordított kód még mindig a helyes memóriacímre ír, csak kis méret- és ciklus-többlettel a Kick Assemblerrel épített ugyanazon forráshoz képest.

UltimateBasic mód

A Visual Assembler tartalmaz egy teljes **Ultimate Basic IDE**-t. Az Ultimate Basic egy modern, fordított BASIC nyelv C64 programok, játékok és demók készítéséhez anélkül, hogy minden műveletet alacsony szintű 6502 assemblyben kellene megírni. A fordító helyben fut, és natív C64 PRG kimenetet generál.

A UB szerkesztő megnyitása

Válaszd a **UB** ikont a fő eszköztáron az Ultimate Basic módra váltáshoz. A választott szerkesztőmód megmarad az alkalmazás újraindításai között. Egy új forrás így kezdődik:

```
color bg 0
color border 0

print "HELLO FROM ULTIMATE BASIC"
```

A UB mód `.ub` forrásfájlokkal dolgozik. A **New**, **Open**, **Save** és **Save As** az aktív UB fülre vonatkozik. Egy `.ub` fájl megnyitása automatikusan aktiválja a hozzá tartozó szerkesztőfület.

Az eszköztár mutatja az aktuális UB munkamappát. Ez a mappa a Block/Expert munkamappától elkülönítve tárolódik. Amíg a UB mód aktív, a **File** → **Set working folder** a UB mappát választja ki; a tooltipje jelzi az aktív hatókört. A UB Open/Save dialógusok innen indulnak, és a nem mentett források ezt használják alapként a relatív `include` és `incbin` utvonalakhoz.

Szerkesztő eszközök

A UB eszköztár ugyanazt a vizuális nyelvet és egyéni tooltipeket követi, mint az Expert mód. Nyújt:

- szintaxiskiemelést az aktuális Ultimate Basic nyelvi referencia alapján;
- sorszámokat, amelyek hosszú fájlloknál is szinkronban maradnak;
- minimapot és szerkesztő zoom vezérlőket; kattints a minimapra az ugráshoz, vagy húzd a viewport-jelölőjét folyamatos görgetéshez;
- Keresést (`Ctrl+F` / `Cmd+F`) az Expert-stílusú keresősávval;
- forrásformázást szerkezet-tudatos behúzással;
- autocomplete-ot parancsokhoz és beépített függvényekhez;

- egy kereshető **Commands** panelt szintaxissal, leírással és használati útmutatóval — a leírások követik az aktuális felületi nyelvet (magyar, angol, spanyol, német, holland), angolra visszaesve mindenhez, ami még nincs lefordítva;
- egymástól függetlenül kapcsolható **Project** és **Commands** panelt, egymás mellett megjelenítve, ha mindkettő engedélyezett;
- egymástól függetlenül kapcsolható és átméretezhető **Build Output** és **Disassembly** panelt.

A Disassembly panelen van egy **Copy** gomb, amely a teljes megjelenített forrást vágólapra másolja. A parancssúgó a beépített fordítót követi: például a `sprite_frame id, data_address [, frame]` egymást követő 64 byte-os sprite frame-ekből választ animációs képet.

A parancslista szándékosan magasság-korlátozott, hogy a parancs-részlet kártya kitölthesse a maradék panel-magasságot. A részlet-terület függetlenül görgethető a hosszabb szintaxis-leírásokhoz.

Projektek, fülek és startup fájlok

Az Ultimate Basic projektek `.proj` fájlokat használnak, és több `.ub` forrást tartalmazhatnak. A Project panel listázza a nyitott fájlokat, jelöli a nem mentett füleket, és mutatja a felfedezett címkéket, függvényeket és szubrutinokat. A projekt-műveletekkel létrehozatsz, megnyithatsz, menthetsz és bezárhatsz egy projektet, vagy hozzáadhatsz egy újabb forrásfájlt.

Kattints egy projektfájl melletti csillagra, hogy **startup fájl**nak jelöld. A Build, Run, D64, C64 Ultimate és Debug parancsok azt a startup forrást fordítják, még akkor is, ha éppen egy másik fül aktív. Startup-választás nélkül az aktív UB fül használdik.

Buildeles és diagnosztika

A **Build** gomb ugyanazt a középre igazított folyamatjelző élményt nyitja meg, mint a többi Visual Assembler futtatási munkafolyamat. A sikeres buildek frissítik a Build Output, Build Info és Disassembly paneleket. Engedélyezd a **Verbose**-t, hogy a fordító memóriatérkép-részletei, belső zero-page allokációi és a generált kód adatai is bekerüljenek.

Ha a fordítás megghiúsul:

- a Build Output automatikusan láthatóvá válik;
- a fordítói hibák pirossal jelennek meg;
- a középre igazított fordítási dialógus megjeleníti a hibát;
- a forrássort tartalmazó hibák kiválasztják azt a sort az aktív UB szerkesztőben.

A Build Info jelenti a load/end címkéket, a kód- és PRG-méreteket, az Exomizer állapotot, a változókat, tömböket, függvényeket/szubrutinokat és címkéket.

Futtatás, D64 és Exomizer

A fő osztott **Run** gomb minden szokásos célponton támogatja az Ultimate Basic-et:

Futtatási mód	Ultimate Basic viselkedés
Run as PRG	Fordítás és a PRG közvetlen indítása VICE-ban.
Run via D64	Fordítás, a szabványos D64 csomagoló dialógus megnyitása, majd a lemez indítása VICE-ban.
Run on Ultimate	A PRG feltöltése és futtatása a beállított C64 Ultimate REST kapcsolaton keresztül.
Run D64 on hardware	D64 csomagolása és küldése a beállított C64 Ultimate-re.

A globális **Settings** → **Exomizer** opció a UB buildekre és a szokásos futtatási célpontokra is vonatkozik; nincs külön UB eszköztár kapcsoló. A debugger indítások szándékosan a tömörítetlen PRG-t használják, hogy a fordítói címek és szimbólumok továbbra is egyezzenek a végrehajtott programmal.

Engedélyezd a **Settings** → **Program Settings** → **Generate UltimateBasic ASM source (.asm)** opciót, hogy a fordító által generált assembly egy PRG vagy D64 build mellé, azonos alapnévvel elmentődjön. Ez egy build opció, így a UB eszköztár nem tartalmaz külön ASM export gombokat. Egy `load "NAME", $address` utasítás megadja a hozzá tartozó D64 extra fájl PRG betöltési címét is.

Debugger szimbólumok és disassembly

A buildek Ultimate Basic debug információt kérnek, és három kompatibilis sidecart állítanak elő:

- `.sym` a KickAssembler-stílusú szimbólumokhoz;
- `.dbg` a C64Debugger/RetroDebugger forrás- és szegmensinformációhoz;
- `.vs` a VICE monitor címkékhez.

A színezett UB Disassembly panel feloldja az ismert címkéket, és megjeleníti a címeket, byte-okat, mnemonikokat és operandusokat. A **Debug** gomb elindítja a RetroDebugget a nyers UB PRG-vel, a debug sidecarokkal és a fordító címkéivel, függvényeivel, szubrutinjaival, változóival és tömbjeivel. A debug wait és unpause beállítások közzé a szokásos Visual Assembler debugger konfigurációval.

Ultimate Basic kézikönyv és forrás

A UB eszköztár könyv-ikonja offline megnyitja a hozzá tartozó Ultimate Basic `MANUAL.pdf`-et; az indítási welcome dialógus manual gombja ugyanazt a kézikönyvet nyitja. A Visual Assembler a fordítót és a PDF-et is a rögzített upstream Git/Cargo függőségből veszi, így az IDE nem tart karban egy második másolatot az Ultimate Basic implementációból. Az About dialógus és a splash képernyő a tényleges függőség-verziót mutatja.

Az Ultimate Basic önálló nyílt forráskódú projektként is elérhető:

<https://github.com/zstarczali/UltimateBasic>

A fordító be van csomagolva a Visual Assemblerbe, így futásidőben nincs szükség külön `ub` futtathatóra.

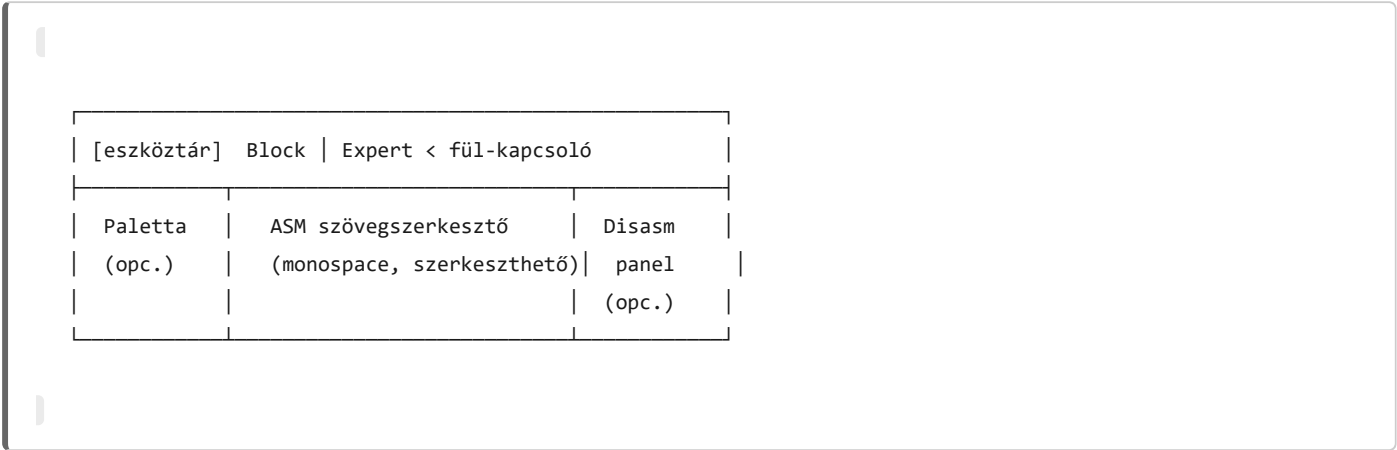
6. Expert mód

Az Expert mód egy teljes értékű, közvetlen szöveges 6502 assembly szerkesztő, amely a blokk-szerkesztő mellett él. Minden fül lehet Blokk vagy Expert módban — bármikor szabadon válthatsz köztük a felső sávban lévő **Block / Expert** kapcsolóval.

Módváltás

- **Block** → **Expert**: az aktuális program szöveggé szerializálódik (soronként egy utasítás, címkék, makrók direktívaként). Az Expert módban végzett szerkesztések visszaszinkronizálódnak a blokk-tömbbe, amikor visszaváltasz vagy egy műveletet indítasz.
- **Expert** → **Block**: a szöveg a `parseAsmText()`-tel parse-olódik, és az eredmény lecseréli a blokk-programot. Ha a parse meghiúsul, fordítási hiba dialógus jelenik meg.
- Az **üres sorok** megőrződnek a round-tripek során: az Expert szerkesztő üres sorai vékony szaggatott elválasztóként jelennek meg Blokk módban, és üres sorként állnak vissza az Expertbe visszaváltáskor.

Szerkesztő elrendezés



Panel	Kapcsoló	Leírás
Paletta	#expert-palette-btn	A bal oldali blokk-paletta — húzz blokkokat a szerkesztőbe, vagy kattints a kurzorhoz beszúráshoz
ASM szerkesztő	mindig látható	Teljes monospace textarea élő szintaxiskiemelés-overlay-jel
Disasm panel	#expert-disasm-btn	Tiszta 6502 disassembly: minden utasítás mutatja a címet, hex byte-okat és numerikus operandusokat; a makrók teljesen kifejtve

Eszköztár gombok

Gomb	ID	Funkció
Format	#expert-format-btn	Forrás automatikus formázása (címkék a 0. oszlopba, 4 szóköz behúzás, 1 szóköz mnemonik/operandus közt)
Load .asm	#expert-load-asm-btn	.asm fájl megnyitása — a tartalom egy új fülbe töltődik, a fájlnev a fül címkéje. Minden betöltött fájl önálló füllé válik saját program-blokkokkal és szerkesztő-állapottal.
Save .asm	#expert-save-asm-btn	A szerkesztő tartalmának mentése .asm fájlba (első mentéskor fájl-dialogus)
Build Info	#expert-build-info-btn	A Build Info dialógus megnyitása (origin, méret, címkék, hibák)
HL	#expert-hl-btn	Szintaxiskiemelés kapcsolása (nagyon nagy fájlknál kikapcsolható)
Autocomplete	#expert-autocomplete-btn	Az expert autocomplete javaslatok be/ki. Kikapcsolva nem jelenik meg direktíva-, mnemonik- vagy címke-popup az expert szerkesztőben.
Region selection	#expert-region-selection-btn	Az automatikus régió-kiemelés kapcsolása Expert módban. A fold-állapot tárolva marad, de kikapcsolva a szerkesztő a teljes forrást láthatóan tartja, és nem választja ki automatikusan az aktuális régiót.
Collapse / expand all regions	#expert-region-fold-all-btn	Minden .region blokk becsukása vagy kinyitása egy kattintással. Ha bármelyik régió nyitva van, a gomb mindet becsukja; ha minden régió már csukva van, a következő kattintás mindet kinyitja. A gomb világít, ha minden régió becsukott.

Gomb	ID	Funkció
Line numbers	#expert-line-numbers-btn	A sorszám-margó kapcsolása a szerkesztő bal oldalán. A margó szinkronban marad a görgetési pozícióval, és élőben frissül gépelés közben.
Find	#expert-find-btn	A lebegő Keresés sáv megnyitása (Ctrl+F). Gépelj a kereséshez; a találatok kiemelődnek az overlay-ben. Enter / Shift+Enter navigál a találatok között. Escape bezárja a sávot.
Zoom out / in	#expert-zoom-out-btn / #expert-zoom-in-btn	A szerkesztő betűmérete csökken / nő (8–28 px). A beállítás megőrződik.
Palette	#expert-palette-btn	A bal oldali mnemonik-paletta megjelenítése/elrejtése
Disasm	#expert-disasm-btn	A disassembly panel megjelenítése/elrejtése (tisztá 6502, makrók kifejtve)
Monitor	#expert-monitor-btn	A monitor hex-dump panel megjelenítése/elrejtése
Minimap	#expert-minimap-btn	A kód-minimap sáv megjelenítése/elrejtése a szerkesztő jobb oldalán

Szerkesztő gyorsbillentyűk: a Ctrl+/ (Cmd+/ macOS-en) megjegyzésbe teszi az aktuális sort vagy minden kijelölt sort; Shift -tel az elől álló megjegyzés-jelölő eltávolítható azokból a sorokból.

Az inline megjegyzések (például LDA \$12 ; magyarázat) az utasítás során maradnak Expert és Blokk mód közti váltáskor. Blokk módban zöld dőlt ; comment jelölőként jelennek meg a blokk fejlécében; a jelölő fölé víve az egeret a teljes szöveg feltárul, ha csonkolt.

Expert szerkesztő minimap

Az Expert szerkesztő minimapja egy keskeny canvas-sáv (88 px) a szerkesztő-terület jobb szélén. Minden forrásor kicsinyített reprezentációját kirajzolja:

Sáv színe	Token típus
Megjegyzés-szín	; -vel kezdődő sorok
Címke-szín	label: definíciót tartalmazó sorok
Direktíva-szín	.byte , .macro , .region és minden más direktíva
Mnemonik-szín	Minden más (utasítások)

Egy félig áttetsző viewport-jelző (kiemelt színű téglalap) mutatja, a forrás mely része látható éppen. Kattints bárhol a minimapra az odaugráshoz; húzd a folyamatos görgetéshez. A minimap függetlenül görget, hogy a viewport-jelzőt középen tartsa. Az állapot a UI beállításokban tárolódik (expertMinimap kulcs).

Hibakiemelés

A fordítani nem sikerülő sorok valós időben pirossal emelődnek ki (árnyalt háttér + bal oldali kiemelő szegély), 350 ms-mal minden billentyűleütés után. Az első hibaüzenet az állapotsávban is megjelenik. Javítsd a sort, és a kiemelés automatikusan eltűnik.

Szintaxiskiemelés

A szerkesztő egy átlátszó `<div>` overlay-t (`expert-h1`) használ, amely színes `<span>` elemekkel tükrözi a textarea tartalmát. A kiemelés a **HL** gombbal kikapcsolható nagyon nagy programoknál a teljesítmény érdekében.

Szín	Token
Sárgászöld	Mnemonikok (<code>LDA</code> , <code>STA</code> , <code>JMP</code> , ...) és long-branch pszeudo-utasítások (<code>LBNE</code> , <code>LBEQ</code> , ...)
Kék	Direktívák (<code>.byte</code> , <code>.word</code> , <code>.fill</code> , <code>.assert</code> , <code>*</code> = , ...)
Narancs	Számok (<code>\$FF</code> , <code>%1010</code> , <code>255</code>)
Cián	Címkék — <code>:</code> -re végződő sorok, beleértve a lokális címkéket (<code>.loop:</code>) és az önmódosító kód operandus-címkéit (<code>value:</code> a <code>LDA value:#\$00</code> -ban)
Türkiz	String literálok
Sötétzöld	Megjegyzések (<code>;</code> ...)

A `REGION` / `ENDREGION` direktívák a többi assembler-direktívához hasonlóan emelődnek ki. A becsukott régiókból csak a régió-fejléc marad látható a szerkesztőben, amíg újra ki nem nyitod. Az új **Region selection** eszköztár-kapcsoló csak az automatikus aktuális-régió kiemelést vezérli Expert módban; kikapcsolva a forrás láthatóan marad a fold-állapot változtatása nélkül.

Forrásformázó

Kattints a **Format** gombra (`#expert-format-btn`) az aktuális forrás automatikus formázásához:

- A címke-definíciók a 0. oszlopba kerülnek.
- Az utasítások 4 szóközzel vannak behúzva.
- A mnemonikok nagybetűsítve.
- Pontosan egy szóköz a mnemonik és az operandus közt (a fölösleges whitespace normalizálódik).
- Ha a forrás már formázott, `"Already formatted"` állapot jelenik meg.

Projekt panel és fülek

Az Expert mód támogat egy **projekt panelt** (`#expert-project-panel`) több-fájlos `.proj` projektekhez:

- A `.proj` fájl egy JSON manifeszt, amely felsorolja a forrásfájlokat és metaadataikat.
- Nyiss meg egy projektet a **Menu** → **File** → **Open project** menüvel, vagy húzz egy `.proj` fájlt az ablakra.
- A projekt minden fájlja külön **fülként** nyílik meg a szerkesztő tetején lévő fül-sávban.
- A **Close Project** (`Menu` → `File` → `Close project` / `#menu-close-project`) egyszerre bezárja az aktuális projektet és minden fájlfülét. A bezárás előtt rákérdez a nem mentett változtatások mentésére. A projekt panel visszaáll üres állapotba, és a `_expertProjectData` törlődik.
- Minden fájl megjelölhető **startup fájlként** (★ csillag ikon). Ha van startup fájl beállítva, a **Run** gomb (PRG, D64, Ultimate) mindig annak a fájlnak a kódját assembleálja és futtatja — függetlenül attól, melyik fül aktív éppen. Ez Blokk és Expert módban is működik.
- A projekt panel alján lévő **szimbólumok** szakasz függőlegesen átméretezhető a fájlfa és a szimbólumlista közti elválasztóval, így a hosszú szimbólumlisták több helyet kaphatnak, amikor szükséges.

Fül-sáv

A fül-sáv a szerkesztő fölött jelenik meg, ha egynél több fül van nyitva.

Funkció	Leírás
Dirty pont	A fül nevén lévő kis kiemelt színű pont nem mentett változtatást jelez
Görgető nyilak	Bal/jobbs görgető gombok jelennek meg, ha több fül van, mint amennyi elér a sávban
Bezárás (x)	Bezárja a fület; ha dirty, rákérdez a mentésre
Fájlkiterjesztés	A teljes fájlnev a kiterjesztéssel együtt (.c64va , .json) látszik
Jobbklikk menü	Jobbklikk egy fülre (vagy üres fül-sáv területre): New tab , Close tab , Close other tabs , Close tabs to the right , Close all tabs . A csoportos bezárás dirty fülönként rákérdez, és megáll, ha megszakítod. A Close all mindig hagy egy üres fület.

Tipp: A paletta-szinkron (`#expert-palette-sync-btn`) szinkronban tartja a paletta-kijelölést a kurzornál lévő mnemonikkal. Kapcsold ki, ha nem szeretnéd, hogy a paletta ugráljon szerkesztés közben.

7. Címzési módok

Minden 6502 utasítás egy vagy több címzési módot támogat. A mód-választó minden blokkon megjelenik.

Mód	Címke	Példa	Leírás
implied	Implied	<code>NOP</code>	Nincs operandus; az utasítás önmagában teljes
immediate	Immediate	<code>LDA #\$FF</code>	Inline konstans; az assembler automatikusan hozzáadja a <code>#</code> -t
zeroPage	Zero page	<code>LDA \$10</code>	Egy byte-os cím a nulladik lapon (0–255)
zeroPageX	Zero page,X	<code>LDA \$10,X</code>	Zero page cím + X regiszter offset (az eredmény a 0. lapon körbefordul)
zeroPageY	Zero page,Y	<code>LDX \$FB,Y</code>	Zero page cím + Y regiszter offset
absolute	Absolute	<code>LDA \$0400</code>	Teljes 16 bites memóriacím
absoluteX	Absolute,X	<code>LDA \$0400,X</code>	16 bites cím + X regiszter offset
absoluteY	Absolute,Y	<code>LDA \$0400,Y</code>	16 bites cím + Y regiszter offset
relative	Relative/Label	<code>BNE loop</code>	Elágazás utasításokhoz; adj meg egy címkenevet vagy célcímet
indirectX	Indirect,X	<code>LDA (\$FB,X)</code>	Zero page indexelt indirekt (operandus = zero page cím, 1 byte)
indirectY	Indirect,Y	<code>LDA (\$FB),Y</code>	Zero page indirekt indexelt (operandus = zero page cím, 1 byte)
indirect	Indirect	<code>JMP (\$0100)</code>	Indirekt; csak JMP-vel használható

Címke-kifejezések operandusként

Bármely operandus-mező, amely címet vagy immediate értéket elfogad, közvetlenül elfogad egy **konstans nevet** is (`CONST` blokkból vagy `LABEL` -ből). Ezen felül **címke+offset** vagy **címke–offset** kifejezésekkel hivatkozatsz egy nevesített konstanshoz képesti címre:

Szintaxis	Példa	Leírás
<code>label</code>	<code>STA screen_ram,X</code>	A címke/konstans értékére oldódik

Szintaxis	Példa	Leírás
<code>label+\$hex</code>	<code>STA screen_ram+\$0100,X</code>	Címke-cím plusz hex offset
<code>label+decimal</code>	<code>STA screen_ram+256,X</code>	Címke-cím plusz decimális offset
<code>label-\$hex</code>	<code>LDA table-\$10</code>	Címke-cím mínusz hex offset
<code>#<label</code>	<code>LDA #<screen_ram</code>	A címke-cím alsó byte-ja
<code>#>label</code>	<code>LDA #>screen_ram</code>	A címke-cím felső byte-ja
<code>*</code>	<code>BNE *</code>	Aktuális program counter (az utasítás saját címe); <code>*</code> -gal az elágazások végtelen önhurkot generálnak (offset <code>\$FE</code>)

Példa — két képernyőlap törlése CONST-tal:

```

; .CONST screen_ram = $0400
    LDX #$00
clear:
    STA screen_ram,X
    STA screen_ram+$0100,X
    DEX
    BNE clear

```

A `*` program counter kifejezésekben

(Új a 2.3.9-ben.) A `*` már nem korlátozódik arra, hogy az egész operandus legyen — megjelenhet **bárhol egy operandus-kifejezésen belül**, és annak az utasításnak a címét jelöli, amelyre írták. Fordítási időben oldódik fel az adott utasítás valódi címéhez képest, így nincs szükség címkére a rövid relatív ugrásokhoz vagy a PC-relatív adatolvasásokhoz.

Szintaxis	Példa	Jelentés
<code>*</code>	<code>BNE *</code>	Ugrás önmagára (végtelen hurok, offset <code>\$FE</code>)
<code>*-n / *+n</code>	<code>BNE *-5 , BEQ *+4</code>	Elágazás az aktuális PC-hez képest <i>n</i> byte-tal
<code>JMP *+n</code>	<code>JMP *+20</code>	Az aktuális PC-ből számított abszolút ugrás
<code>#<* / #>*</code>	<code>LDA #<*, LDA #>*</code>	Az aktuális PC alsó / felső byte-ja
<code>#>(*+n)</code>	<code>LDA #>(*+63)</code>	Egy PC-relatív cím alsó / felső byte-ja

PC vs. szorzás. A `*` csak akkor számít program counternek, ha *érték-pozícióban* áll — a kifejezés elején, vagy közvetlenül egy operátor, `( , , < , >` vagy whitespace után. A szám, `)` vagy azonosító után álló `*` közönséges szorzás, így a `LDA table*2` és a `CONST_A*4` változatlan.

Hol működik. Bármely operandusban, amely már elfogad kifejezést: elágazási célpontok, `JMP / JSR , LDA / STA /...` abszolút és indexelt, immediate alsó/felső-byte operátorok, és

a `.assert` kifejezése. A `*` sosem változtatja meg egy utasítás méretét, így minden címzési módban biztonságos.

Lokális (pontos) címkék

(Új a 2.3.9-ben.) Az a címke, amelynek a neve ponttal kezdődik — `.loop`, `.skip`, `.done` — **lokális címke**. A legközelebbi előtte lévő **globális** (nem pontos) címke hatókörébe tartozik, és belsőleg `<globális>.<név>` lesz belőle. Két azonos rövid nevű lokális címke különböző globális címkék alatt **nem** ütközik.

```
DrawSprite:
    LDX #0
.loop:          ; == DrawSprite.loop
    LDA SpriteData,X
    STA $2000,X
    INX
    CPX #63
    BNE .loop    ; a DrawSprite-on belül oldódik fel
    RTS

ClearScreen:
    LDX #0
.loop:          ; == ClearScreen.loop – nincs ütközés
    STA $0400,X
    INX
    BNE .loop
    RTS
```

- Egy hatókörön belül `.név` -ként hivatkozz a lokális címkére.
- Másik hatókörből explicit módon `Globális.név` -ként (pl. `JMP ClearScreen.loop`).
- Az első globális címke előtt írt `.név` sima top-level `.név` marad.
- A lokális címkék változatlanul round-trip-elnek Blokk $\rightleftharpoons$ Expert mód közt; a `<globális>.` prefix csak layout-idejű részlet, sosem tárolódik a blokk-programban.

Önmódosító kód operandus-címkék

(Új a 2.3.9-ben.) Tedd egy utasítás operandusa elé a `címke:` prefixet, hogy a címke az **operandus-byte-ra** kerüljön, ne az opcode-ra. Az utasítás abból fordul, ami a kettőspont után áll.

```
setup:
    LDA value:#$00    ; 'value' -> a #$00 operandus-byte címe
    ...
patch:
```

LDA #new
STA value ; közvetlenül az operandus-byte-ot írja – klasszikus SMC

A value az <utasítás címe> + 1 -re mutat (az első operandus-byte), minden címzési módban.
Ez leváltja a régebbi STA instruction+1 / instruction: LDA #\$00 mintát. Round-trip-el
Blok k Expert mód közt (az operandus-mező megtartja a címke: prefixet).

8. Szabványos 6502 utasítások

Data Movement

Mnemonik	Leírás	Módok
LDA	Akkumulátor betöltése	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
LDX	X regiszter betöltése	immediate, zeroPage, zeroPageY, absolute, absoluteY
LDY	Y regiszter betöltése	immediate, zeroPage, absolute, absoluteX
STA	Akkumulátor tárolása	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
STX	X regiszter tárolása	zeroPage, zeroPageY, absolute
STY	Y regiszter tárolása	zeroPage, absolute

Arithmetic

Mnemonik	Leírás	Megjegyzés
ADC	Összeadás carryvel	A legtöbb esetben állítsd be a carryt SEC -kel használat előtt
SBC	Kivonás carryvel	Állítsd be a carryt SEC -kel kivonás előtt
INC	Memória növelése	—
DEC	Memória csökkentése	—
CMP	Összehasonlítás A-val	Flag-eket állít; nem módosítja A-t
CPX	Összehasonlítás X-szel	—
CPY	Összehasonlítás Y-nal	—

Logic

Mnemonik	Leírás
AND	Logikai ÉS az akkumulátorral
ORA	Logikai VAGY az akkumulátorral
EOR	Kizáró VAGY az akkumulátorral

Mnemonik	Leírás
BIT	Bitek tesztelése a memóriában A ellenében (N, V, Z flag-eket állít)

Jumps & Branches

Mnemonik	Leírás
JMP	Feltétel nélküli ugrás (abszolút vagy indirekt)
JSR	Ugrás szubrutinba (a visszatérési címet a stackre menti)
RTS	Visszatérés szubrutinból
RTI	Visszatérés megszakításból
BNE	Ugrás, ha nem egyenlő (Z=0)
BEQ	Ugrás, ha egyenlő (Z=1)
BCC	Ugrás, ha carry törölve (C=0)
BCS	Ugrás, ha carry beállítva (C=1)
BMI	Ugrás, ha negatív (N=1)
BPL	Ugrás, ha pozitív (N=0)
BVC	Ugrás, ha overflow törölve (V=0)
BVS	Ugrás, ha overflow beállítva (V=1)

LBNE / LBEQ / ... (Hosszú ugrások)

(Új a 2.3.9-ben.) A **Hosszú ugrások** paletta-kategória nyolc pszeudo-utasítást tartalmaz, amelyek feltételes elágazásként viselkednek, de **bármely címet** elérnek, nem csak -128...+127-et. Mindegyik egy invertált elágazásra fordul, amely átugrik egy 3 byte-os `JMP`-t — mindig **5 byte**:

```
LBNE done      ; erre fordul:  BNE *+3   ($D0 $03)
                ;                JMP done  ($4C lo hi)
```

Hosszú op	Feltétel	Kibocsátott forma
LBNE	nem egyenlő (Z=0)	BEQ *+3 / JMP target
LBEQ	egyenlő (Z=1)	BNE *+3 / JMP target
LBCC	carry törölve (C=0)	BCS *+3 / JMP target
LBCS	carry beállítva (C=1)	BCC *+3 / JMP target
LBMI	negatív (N=1)	BPL *+3 / JMP target

Hosszú op	Feltétel	Kibocsátott forma
LBPL	pozitív (N=0)	BMI *+3 / JMP target
LBVC	overflow törölve (V=0)	BVS *+3 / JMP target
LBVS	overflow beállítva (V=1)	BVC *+3 / JMP target

- Operandus: címke, * -kifejezés vagy literál cím — ugyanaz, mint egy normál elágazási célpont.
- Költség: 5 byte és 1 extra ciklus a végrehajtott ágon egy rövid elágazáshoz képest. Nincs automatikus előléptetés rövid elágazásból — te választod az LBxx -et explicit módon.
- Amikor egy sima elágazás (BNE , BEQ , ...) tartományon kívülre esik, a fordítói hiba mostantól megnevezi a pontos túllépést, és javasolja a megfelelő LBxx -et.

Register Operations

Mnemonik	Leírás
TAX	Átvitel A → X
TAY	Átvitel A → Y
TXA	Átvitel X → A
TYA	Átvitel Y → A
TSX	Átvitel stack pointer → X
TXS	Átvitel X → stack pointer
INX	X növelése
DEX	X csökkentése
INY	Y növelése
DEY	Y csökkentése

Shift & Rotate

Mnemonik	Leírás
ASL	Aritmetikai balra tolás
LSR	Logikai jobbra tolás
ROL	Balra forgatás a carryn keresztül
ROR	Jobbra forgatás a carryn keresztül

Stack

Mnemonik	Leírás
PHA	Akkumulátor push a stackre
PHP	Processzor státusz push a stackre

Mnemonik	Leírás
PLA	Akkumulátor pull a stackről
PLP	Processzor státusz pull a stackről

System / Flags

Mnemonik	Leírás
CLC	Carry flag törlése
CLD	Decimal mód törlése
CLI	Interrupt tiltás törlése
CLV	Overflow flag törlése
SEC	Carry flag beállítása
SED	Decimal mód beállítása
SEI	Interrupt tiltás beállítása
NOP	Nincs művelet
BRK	Kényszerített break / szoftveres megszakítás

Illegal / dokumentálatlan utasítások

Ezek haladó használatra támogatottak. Óvatosan használd — a viselkedés chipenként eltérhet.

LAX , SAX , DCP , ISC , SLO , RLA , SRE , RRA , ANC , ALR , ARR , AXS

9. Makró blokkok — referencia

A makró blokkokkal gyakori feladatokat végezhetsz el egy lépésben — ahelyett, hogy 10–20 utasítást írnál kézzel, leteszel egy blokkot, és az assembler generálja neked a kódot. Gondolj rájuk beépített szubrutinként.

LABEL

Mint egy **sorszám BASIC-ben** — de szám helyett névvel. Ugrási célpont a `JMP` , `JSR` , `BNE` , stb. számára.

Mező	Leírás
Címkenév	Azonosító, amit a <code>JMP</code> , <code>JSR</code> , <code>BNE</code> , stb. használ

Expert szintaxis:

```
loop:
```

Generált ASM:

```
loop:  ; $0820
```

Az aktuális cím megjegyzésként jelenik meg. A címkék **0 byte méretűek**.

COMMENT

Mint a **REM BASIC-ben** — jegyzet magadnak, amit az assembler teljesen figyelmen kívül hagy.

Expert szintaxis:

```
; Ide jön a megjegyzés szövege
```

Generált ASM:

```
; Ide jön a megjegyzés szövege
```

BYTE

Mint a **DATA BASIC-ben** — nyers byte-értékek listáját tárolja inline a programban.

Mező	Leírás
Operandus	Vesszővel elválasztott byte-értékek (pl. \$01, \$02, \$FF vagy 1, 2, 255)

Expert szintaxis:

```
.byte $01, $02, $FF
```

Generált ASM:

```
.byte $01, $02, $FF
```

Lo/hi byte címke-hivatkozások: a BYTE elfogadja a KickAssembler / ca65 stílusú `<label` (alsó byte) és `>label` (felső byte) tokeneket a numerikus értékek mellett. Az assembler fordítási időben feloldja a címke címét, és beszúrja a megfelelő byte-ot. Példa:

```
.byte <frame_0, >frame_0, <frame_1, >frame_1
```

Ez a `frame_0` címének alsó byte-ját tárolja, majd a felsőt, majd ugyanezt a `frame_1` -re. Hasznos jump table-ök és címlisták építéséhez.

Méret: a lista byte-jainak száma.

WORD

Mint a **DATA BASIC-ben, de 16 bites számokhoz**. Minden érték két byte-ként tárolódik (alsó byte először, majd felső byte — 6502 little-endian sorrend).

Mező	Leírás
Operandus	Vesszővel elválasztott 16 bites értékek (pl. <code>\$0400</code> , <code>\$C000</code>)

Expert szintaxis:

```
.word $0400, $C000
```

Generált ASM:

```
.word $0400, $C000
```

Méret: 2 byte szavanként.

FILL

Mint a `FOR I=1 TO N : POKE addr+I, val : NEXT` — egy memóriablokkot tölt fel ugyanazzal a byte-tal, de egyetlen blokkban. Kiváló területek törlésére vagy táblák előfeltöltésére.

Mező	Leírás
Operandus	<code>count,value</code> — pl. <code>256,0</code> 256 byte-ot tölt fel nullával

Expert szintaxis:

```
.fill 256, $00
```

Generált ASM:

```
.fill 256, $00
```

Kifejezés-szintaxis: a `count` és a `value` is elfogad aritmetikai kifejezéseket. Hivatkozhat sz CONST nevekre, használhat hex/bináris literálokat, és hívhat beépített matematikai függvényeket:

Kifejezés	Jelentés
<code>TILE_COUNT, \$00</code>	count egy CONST-ból, value hex literál
<code>40*25, 0</code>	inline szorzás
<code>round(sin(PI/4)*255), \$80</code>	trigonometria

Beépített függvények: `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, `PI` konstans

Operátorok: `+` `-` `*` `/` Literálok: `$FF` (hex), `%10110000` (bináris) Alsó/felső byte: `lo(expr)`, `hi(expr)`

Méret: a count érték byte-ban.

ALIGN

Előrébb csúsztatja az aktuális címet a következő tiszta határra nulla kitöltő byte-ok beszúrásával. A C64 megköveteli, hogy a sprite adat 64 byte-os határon kezdődjön — az `ALIGN 64` ezt automatikusan intézi.

Mező	Leírás
Határ	Igazítási érték — pl. <code>64</code> (sprite határ), <code>256</code> (lap), <code>\$2000</code> (bitmap)

Expert szintaxis:

```
.align 64
.align $2000
```

Generált ASM:

```
; ALIGN 64 → $0840 (12 byte kitöltés)
```

Méret: dinamikus — az aktuális program counter pozíciójától függ.

Tipp: használj `ALIGN 64` -et a sprite adat előtt, `ALIGN 256` -ot a lap-igazított táblákhoz.

TEXT

Mint a **PRINT AT** — szöveget ír közvetlenül a C64 képernyőjére adott oszlopban és sorban, a KERNAL használata nélkül. Karakterenként egy LDA/STA párt generál, a `$0400` -nál lévő screen RAM-ot célozva.

Mező	Leírás
Text	A megjelenítendő string
X	Oszlop (0–39)
Y	Sor (0–24)
Label (opcionális)	Címkét rendel, amely a kiszámolt képernyőcímre mutat
Lowercase charset	Jelölőnégyzet — lásd lentebb

Karakterkészlet módok:

A C64-nek két, futásidőben választható karakterkészlete van:

Mód	\$D018 bit 1	Nagybetűs bemenet	Kisbetűs bemenet
Nagybetű/grafika (alap)	0	A – Z → screen code \$01–\$1A ✓	szintén nagybetűként kezelve
Kisbetű/nagybetű (CHARSET lower után)	1	A – Z → \$01–\$1A (nagybetű)	a – z → \$41–\$5A (kisbetű) ✓

- **Nagybetűs karakterkészlet (alap, jelölőnégyzet üres):** azt írd, amit nagybetűvel akarsz látni. A "HELLO" HELLO -ként jelenik meg. A kisbetűs bemenet nagybetűs screen code-okra képződik.
- **Kisbetűs karakterkészlet (jelölőnégyzet bepipálva):** pontosan azt a kis/nagybetűzést írd, amit látni akarsz. "hello" → kisbetűs megjelenítés, "HELLO" → nagybetűs. Futásidejű karakterkészlet-váltást igényel a képernyőre írás előtt (használd a **CHARSET lower** makrót).

Generált ASM (nagybetűs mód, "HELLO"):

```
LDA #$08      ; 'H' screen code $08
STA $0400
LDA #$05      ; 'E' screen code $05
STA $0401
...
```

Expert szintaxis:

```
.text 0, 2, "HELLO"      ; nagybetűs karakterkészlet (alap)
.text 0, 2, "hello", lower ; kisbetűs karakterkészlet
```

A karakterek **screen code-ként** kódolódnak (nem PETSCII). **Méret:** text.length × 5 byte (LDA + STA karakterenként).

STRING

Mint egy string **POKE-olása** bármely memóriacímre futásidőben. LDA/STA párokat generál, amelyek minden karakter screen code-ját egymást követő címekre másolják.

Mező	Leírás
Text	A kiírandó string
Address	Cél memóriacím — \$C000 hex vagy címkenév
Label (opcionális)	Címkét rendel, amely a célcímre mutat
Shift	Hex érték (00–FF), amely minden screen code byte-hoz hozzáadódik (pl. \$80 = inverz)
Lowercase charset	Jelölőnégyzet — ugyanaz a szemantika, mint a TEXT-nél (lásd a TEXT szakaszt)

Expert szintaxis:

```
.string $C000, "HELLO"           ; nagybetűs karakterkészlet (alap)
.string $C000, "hello", lower    ; kisbetűs karakterkészlet
.string $C000, "HELLO", 80       ; shifttel (inverz)
.string $C000, "hello", 80, lower ; shift + kisbetű
.string $C000, "HELLO" :my_string ; macroLabel-lel
```

Generált ASM:

```
LDA #$08      ; 'H' screen code
STA $C000
LDA #$05      ; 'E' screen code
STA $C001
...
```

A karakterek **screen code-ként** kódolódnak (nem PETSCII). Az opcionális **Shift** érték minden byte-hoz hozzáadódik, pl. **\$80** az inverz videóhoz. **Méret:** `text.length × 5` byte (egy LDA + egy STA karakterenként).

DATA

Mint egy **POKE hurok** — nyers byte-ok listáját írja egy memóriacímre futásidőben, byte-onként egy LDA/STA párral.

Mező	Leírás
Bytes	Vesszővel elválasztott byte-értékek
Address	Cél memóriacím — <code>\$C000</code> hex vagy címkenév
Label (opcionális)	Címkét rendel, amely a célcímre mutat

Expert szintaxis:

```
.data $C000, $01, $02, $03      ; hex cím
.data my_buf, $01, $02, $03     ; címke-cím
.data $C000, $01, $02, $03 :mydata ; macroLabel-lel
```

Generált ASM:

```
LDA #$01
STA $C000
LDA #$02
STA $C001
...
```

Méret: `byte_count` × 5 byte (egy LDA + egy STA byte-onként).

RAWBYTES

Mint a **DATA**, amely közvetlenül a memóriába töltődik — egyáltalán nincs futásidejű kód. A byte-ok a PRG betöltésének pillanatától jelen vannak, még mielőtt a kódod elindulna. Ezt használd sprite adathoz, pályatérképekhez, lookup táblákhoz, bármihez, aminek csak egy konkrét címen kell lennie.

Mező	Leírás
Bytes	Vesszővel elválasztott byte-értékek
Address	Cél memóriacím — <code>\$C000</code> hex vagy címkenév
Label (opcionális)	Címkét rendel, amely a célcímre mutat

Expert szintaxis:

```
.rawbytes $C000, $00, $00, $00 ; hex cím
.rawbytes sprite_data, $00, $00 ; címke-cím
.rawbytes $0C50, $00, $00 :nev ; macroLabel-lel – más kód használhat LDA nev,X-et
```

Méret a kódban: 0 byte. Az adat a megadott címre kerül a kimenetben.

DATA vs RAWBYTES: a DATA LDA/STA kódot generál, amely futásidőben másolja a byte-okat (lassabb, de működik, ha az adatnak dinamikusnak kell lennie). A RAWBYTES egyszerűen elhelyezi a byte-okat — nincs kód, azonnali, nulla költség.

RAWTEXT

Mint a RAWBYTES, de szöveghez — a stringet screen code-ként kódolja, és a byte-okat fix címre helyezi **futásidejű kód nélkül**. A szöveg a PRG betöltésének pillanatában készen áll a memóriában.

Mező	Leírás
Text	A kódolandó string

Mező	Leírás
Address	Cél memóriacím — <code>\$C000</code> hex vagy címkenév
Label (opcionális)	Címkét rendel, amely a célcímre mutat
Shift	Hex érték (00–FF), amely minden screen code byte-hoz hozzáadódik (pl. <code>\$80</code> = inverz)
Lowercase charset	Jelölőnégyzet — ugyanaz a szemantika, mint a TEXT-nél (lásd a TEXT szakaszt)

Expert szintaxis:

```
.rawtext $C000, "HELLO"           ; nagybetűs karakterkészlet (alap)
.rawtext $C000, "hello", lower    ; kisbetűs karakterkészlet
.rawtext $C000, "HELLO", 80       ; shifttel (inverz)
.rawtext $C000, "hello", 80, lower ; shift + kisbetű
.rawtext $0400, "HELLO" :my_text  ; macroLabel-lel
```

Generált ASM:

```
; .rawtext "HELLO" -> $C000
; $C000
    .byte $08, $05, $0C, $0C, $0F ; H E L L O (nagybetűs screen code-ok)

; .rawtext "hello", lower -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F ; h e l l o (kisbetűs screen code-ok, $41-$5A tartomány)
```

Méret a kódban: 0 byte. Az adat a megadott címre kerül a kimenetben.

STRING vs RAWTEXT: a STRING LDA/STA kódot generál, amely futásidőben másolja a szöveget. A RAWTEXT betöltéskor beépíti a byte-okat a PRG-be — nincs kód, nincs várakozás.

PETSCII

Mint a **RAWBYTES, de KERNAL kimenethez** — a stringet PETSCII byte-ként kódolja (kompatibilis a `$FFD2` -nél lévő CHROUT-tal), és fix címre helyezi futásidejű kód nélkül. Ezt használd, amikor karaktereket akarsz kiírni `JSR $FFD2` -vel egy hurokban; az új `PRINT` makró ugyanezt a kódolót és kisbetűs jelölőnégyzet-viselkedést használja.

PETSCII vs screen code-ok: a PETSCII és a screen code-ok két különböző kódolás. A screen code `$01` = az A betű; a PETSCII `$41` = az A betű (CHROUT-on át). PETSCII-t csak KERNAL-on keresztüli nyomtatáshoz használj; a screen RAM-ba közvetlen íráshoz használj TEXT/STRING/RAWTEXT-et.

Mező	Leírás
Text	PETSCII byte-ként kódolandó string
Address	Cél memóriacím — \$C000 hex vagy címkenév
Label (opcionális)	Címkét rendel, amely a célcímre mutat
Lowercase PETSCII	Jelölőnégyzet — lásd lentebb

Karakterkészlet módok:

Mód	Nagybetűs bemenet (A – Z)	Kisbetűs bemenet (a – z)
Nagybetű (alap, üres)	\$41 – \$5A (PETSCII nagybetű CHROUT-on át)	szintén \$41 – \$5A -ra képezve
Kisbetű (bepipálva)	Az alfabetikus betűk átképződnek, hogy a látható kis/nagybetűzés konzisztens maradjon a kisbetű/nagybetű karakterkészleten	Ugyanaz a szabály

Expert szintaxis:

```
.petscii $C000, "HELLO"           ; nagybetűs PETSCII (alap)
.petscii $C000, "hello", lower    ; kisbetűs PETSCII ($61-$7A)
.petscii $C000, "HELLO", null     ; null lezáróval
.petscii $C000, "hello", lower, null ; kisbetű + null lezáró
.petscii $C000, "HELLO" :my_msg   ; macroLabel-lel
```

Generált byte-ok (nagybetű, "HELLO"):

```
; $C000
.byte $48, $45, $4C, $4C, $4F ; H E L L O (PETSCII $41-$5A tartomány)
```

Méret a kódban: 0 byte. Az adat a célcímre kerül halasztott adat-szegmensként (mint a RAWBYTES-nál).

Null lezáró: pipáld be a „\$00 hozzáfűzése (null lezáró)” jelölőnégyzetet, hogy automatikusan egy \$00 byte kerüljön a szöveg után. Ideális null-lezárt hurkokhoz:

```
LDX #$00
loop:
LDA msg,X
BEQ done ; a $00 leállítja a hurkot
JSR $FFD2
INX
```

```
BNE loop
done:
RTS
```

Kódolási szabályok:

Bemenet	Nagybetűs mód	Kisbetűs mód
A – Z	\$41 – \$5A	\$61 – \$7A
a – z	\$41 – \$5A (kényszerített nagybetű)	\$41 – \$5A
Szóköz, számjegyek, írásjelek (32–126)	változatlanul	változatlanul
Sortörés	\$0D (RETURN)	\$0D
Egyéb	\$20 (szóköz)	\$20

Tipp: PETSCII-t olyan adathoz használj, amelyet CHROUT-on (\$FFD2) keresztül fogsz kiírni. A screen RAM-ba közvetlen íráshoz inkább STRING-et vagy RAWTEXT-et.

CHARSET

A VIC-II karakter-ROM-ot váltja nagybetű/grafika mód (C64 alap) és kisbetű/nagybetű mód közt, a \$D018 1. bitjének futásidejű módosításával.

Mező	Leírás
Mód	Lowercase — engedélyezi a kisbetű/nagybetű karakterkészletet; Uppercase — visszaállítja az alapértelmezett nagybetű/grafika karakterkészletet

Expert szintaxis:

```
.charset lower      ; váltás kisbetűs karakterkészletre
.charset upper      ; vissza a nagybetű/grafika karakterkészletre
```

Generált ASM:

Kisbetűs mód:

```
LDA $D018
ORA #$02      ; 1. bit be → kisbetű/nagybetű ROM $1800-nál
STA $D018
```

Nagybetűs mód:

```
LDA $D018
AND #$FD      ; 1. bit törlése → nagybetű/grafika ROM $1000-nál
STA $D018
```

Méret: 8 byte (LDA abs + ORA/AND imm + STA abs).

Miért ORA/AND közvetlen írás helyett? A `$D018` a screen RAM helyét is vezérli (7–4. bitek). Csak az 1. bit kapcsolása megőrzi a regiszter többi részét.

Tipikus munkafolyamat:

```
CHARSET lower      ; váltás kisbetűs karakterkészletre
TEXT 0, 0, "hello world" ; [jelölőnégyzet: Lowercase charset]
...
CHARSET upper      ; végén állítsd vissza az alapot
```

Vagy expert módban:

```
.charset lower
.text 0, 0, "hello world", lower
.charset upper
```

Expert módban a `.charset` blokk mostantól a mód-legördülön keresztül is round-trip-el, így a blokk-előnézet és az exportált forrás szinkronban marad.

Megjegyzés: a CHARSET makró csak a VIC karakter-ROM pointerét változtatja. Nem hívja a `$E544`-et (KERNAL charset init). A legtöbb esetben ez elég; csak akkor hívj előtte `JSR $E544`-et, ha azt akard, hogy a KERNAL saját print rutinjai is figyeljék a változást.

CHARDEF

Egyetlen 8×8-as egyéni karaktert definiál egy RAM-alapú karakterkészletben. Inline futásidejű kódot bocsát ki, amely 8 byte-ot másol a `base + index * 8` címre végrehajtáskor — nincs szükség előtte címkére vagy `ORG`-ra.

Mező	Leírás
Charset base	A RAM karakterkészlet báziscíme (alap <code>\$3800</code>). Igazítottnak kell lennie, hogy a VIC-II lássa (lásd lentebb).

Mező	Leírás
Character index	Melyik karakter-slotot definiáljuk újra, 0–255. 65 = 'A' az alapértelmezett screen code elrendezésben.
8 byte	Vesszővel elválasztott bitmap sorok, fentről lefelé. Minden byte 7. bitje = a legbaloldalibb pixel.

Expert szintaxis:

```
.chardef $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00
```

Generált ASM (8 × LDA #b / STA target+n , összesen 40 byte):

```
LDA #$18
STA $3A08
LDA #$3C
STA $3A09
LDA #$66
STA $3A0A
LDA #$7E
STA $3A0B
LDA #$66
STA $3A0C
LDA #$66
STA $3A0D
LDA #$66
STA $3A0E
LDA #$00
STA $3A0F
```

Célcím: \$3800 + 65 * 8 = \$3A08 . Fordítási időben számolódik, és be van égetve az STA operandusokba.

Méret: 40 byte karakterenként (8 × 5).

Tipikus munkafolyamat:

```
; VIC-II a $3800-nál lévő RAM charsetre mutasson
LDA $D018
AND #$F1
ORA #$0E ; 3-1. bitek = %111 → charset a bank+$3800-nál
STA $D018

CHARDEF $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00 ; 'A' újradefiniálása
```

```
CHARDEF $3800, 66, $7C,$66,$66,$7C,$66,$66,$7C,$00 ; 'B' újradefiniálása
; ... további CHARDEF hívások minden egyéni karakterhez
```

Mikor CHARDEF vs alternatívák:

Megközelítés	Használd, ha
CHARDEF	Néhány egyéni karakter kell (mondjuk 1–20). Futásidejű költség 40 byte darabonként.
RAWBYTES @ \$3800	Teljes egyéni karakterkészletet van (256 karakter). Összesen 2 KB adat, nincs futásidejű másolás.
INCBIN "charset.bin" @ \$3800	Külső karakterkészlet-fájl (a Character Editorral építve). A legtisztább opció.
Charset Canvas + INCBIN	Teljes 256 karakteres bitmap egyetlen 128×128-as képként megfestve.

Igazítási emlékeztető: a VIC-II a charset bázist \$0800 többszörösén várja. Érvényes bankok: \$0000 , \$0800 , \$1000 , ... , \$3800 (az aktuális 16 KB-os VIC bankon belül). A RAM charsetek jellemzően a \$2000 , \$2800 , \$3000 vagy \$3800 címen élnek.

BOX_HIT

Tengely-igazított bounding-box (AABB) ütközésteszt két, 4 byte-os zero-page struktúrával leírt téglalap közt. Az eredményt az akkumulátorban adja vissza: **A = 1** átfedésnél, **A = 0** egyébként. Tiszta inline assembly, nincs szubrutinhívás.

Mező	Leírás
Box1 ZP cím	Az első box 4 byte-os struktúrájának zero-page bázisa (alap \$FB).
Box2 ZP cím	A második box 4 byte-os struktúrájának zero-page bázisa (alap \$F7).

Struktúra elrendezés (4 byte boxonként, előjel nélküli 8 bites koordináták):

Offset	Mező
+0	Left
+1	Top
+2	Right
+3	Bottom

Expert szintaxis:

```
.box_hit $FB, $F7
```

Generált ASM (30 byte, teljesen PC-relatív — nincs szubrutin, nincs abszolút ugrás):

```
LDA $FD      ; b1 right
CMP $F7      ; b2 left
BCC no       ; R1 < L2 → nincs találat
LDA $F9      ; b2 right
CMP $FB      ; b1 left
BCC no
LDA $FE      ; b1 bottom
CMP $F8      ; b2 top
BCC no
LDA $FA      ; b2 bottom
CMP $FC      ; b1 top
BCC no
LDA #$01     ; találat
BNE done     ; feltétel nélküli (A ≠ 0)
no: LDA #$00
done:
```

Méret: 30 byte.

Tipikus munkafolyamat:

```
; Játékos sprite box a $FB..$FE címen
LDA sprite_x      ; L
STA $FB
LDA sprite_y      ; T
STA $FC
CLC
ADC #23           ; +23 → B (24 pixel magas sprite)
STA $FE
LDA $FB
CLC
ADC #23           ; R
STA $FD

; Ellenség box a $F7..$FA címen (hasonlóan feltöltve)
; ...

BOX_HIT $FB, $F7   ; játékos vs ellenség teszt → A = 0 vagy 1

CMP #$01
BNE no_collision
JSR handle_hit
no_collision:
```

Megkötések:

- Mindkét zero-page címnek `≤ $FC` -nek kell lennie (minden box 4 egymást követő byte-ot igényel: `zp` , `zp+1` , `zp+2` , `zp+3`).
- A koordináták **előjel nélküli 8 bitesként** kezelődnek (0–255). Ezen tartományon kívüli előjeles sprite-koordinátáknál tárolás előtt normalizálj.
- A két box átfedhet a ZP térben, ha akarod, de általában 8 különálló byte-ot szeretnél.

Miért nem szubrutin? Az inline generálás elkerüli a JSR/RTS overhead-et (14+ ciklus), és a tesztet a cache-ben tartja a szoros game hurkokhoz. Ha sok párt kell tesztelned, tegyél kézzel egy saját `JSR box_hit_sub` -ot egyetlen BOX_HIT blokk köré.

Összehasonlítás a UB `box_hit()` függvényével: az Ultimate Basic ugyanezt a 6502 logikát csomagolja futásidejű függvényként, amely egy változóba tér vissza. VA-ban a BOX_HIT-et inline helyezed oda, ahol a tesztre szükség van; az eredmény az `A` -ban van.

INCBIN

Mint a **BLOAD BASIC-ben** — beemel egy külső binárisfájl (`.bin` , `.prg` , `.sid` , `.raw`), és közvetlenül beágyazza a lefordított PRG-be a megadott címre.

Mező	Leírás
File	Tallózz egy <code>.bin</code> , <code>.prg</code> , <code>.sid</code> vagy <code>.raw</code> fájl kiválasztásához
Address	Cél betöltési cím (pl. <code>\$C000</code>)

Expert szintaxis:

```
.incbin "music.bin", $C000
```

Generált ASM megjegyzés:

```
; INCBIN "music.bin" @ $C000 (2048 byte)
.byte $01, $02, ...
```

Méret a kódban: 0 byte (halasztott adat-szegmens). A bináris a megadott címre ágyazódik.

SID

Mint a **BLOAD zenéhez** — betölt egy `.sid` fájlt a PRG-be, és automatikusan kiolvassa az Init és Play címeket a fejlécből. Hívd az Init-et egyszer induláskor, majd hívd a Play-t az IRQ kezelődből minden frame-ben.

Mező	Leírás
File	Tallózz egy <code>.sid</code> fájl kiválasztásához
Custom address (opcionális)	Felülírja a SID natív betöltési címét (pl. <code>\$1000</code>). Hagyd üresen a SID fejlécből származó cím használatához.

A blokk megjeleníti:

- **Title / Author** a SID fejlécből
- **Load address** — hova kerül az adat a memóriában (a felülírás utáni tényleges cím)
- **Init address** — ezt hívd JSR-rel a zene inicializálásához (relokáció esetén korrigálva, ha egyéni címet használsz)
- **Play address** — ezt hívd JSR-rel minden frame-ben egy IRQ kezelőben (relokációhoz korrigálva)
- Egy **(relocated)** jelvény jelenik meg, amikor egy egyéni cím elmozdítja az adatot az eredeti pozíciójából

Expert szintaxis:

```
.sid "Ikari_Warriors.sid"
.sid "Ikari_Warriors.sid", $1000
```

Generált ASM megjegyzés:

```
; SID "Ikari_Warriors.sid" @ $1000 Init:$1000 Play:$1006 (4096 byte)
```

Méret a kódban: 0 byte inline. A SID bináris a megadott címre kerül halasztott chunkként a PRG-ben.

Fontos: a legtöbb SID fájl beégetett belső abszolút címeket tartalmaz. Csak akkor relokálhatók, ha az egész bináris ugyanazzal az offszettel toódik el. Ha egy SID belső ugrásokat tartalmaz `$10xx` -re, akkor `$1000` -en kell maradnia — más címre mozgatva ezek a belső hivatkozások elromlanak.

Tipikus használat: tegyél egy ORG blokkot a SID blokk elé a címének beállításához. Hívd az Init-et egyszer induláskor, majd hívd a Play-t minden frame-ben egy raszter IRQ kezelőből.

INCLUDE

Mint a **MERGE BASIC-ben** — beemel egy másik fájlt, és a blokkjait inline kifejti ezen a pozíción. Kiváló újrahasználgató szubrutin-könyvtárakhoz. A beemelt blokkok csak olvashatók az aktuális projektben.

Kétféle fájl támogatott:

- **Visual Assembler projekt** (`.json`) — a projekt blokkjai változatlanul beszűrődnek.

- **Sima assembly forrás** (`.inc` , `.asm` , `.s`) — a fájl szöveggént olvasódik, és ugyanúgy parse-olódik, mint az expert mód. Minden fordításkor a fájl újraolvasódik a lemezről (az igazság forrása = a fájl), így bármilyen szerkesztővel külsőleg szerkesztheted.

Mező	Leírás
File	Tallózz egy <code>.json</code> projekt vagy <code>.inc</code> / <code>.asm</code> / <code>.s</code> assembly forrás kiválasztásához
Load address (opcionális)	Ha beállított (hex, pl. <code>C000</code>), a beemelt blokkok arra a címre kerülnek — egy szintetikus <code>ORG</code> szúródik eléjük, felülírva a beemelt fájlban belüli bármely <code>ORG</code> blokkot. Hagyd üresen, hogy a beemelt fájl saját <code>ORG</code> blokkjai vezéreljék az elhelyezést.

Expert-mód szintaxis:

```
.include "library.json"
.include "macros.inc", $1500
.include "sprites.asm"
```

- A **fájlkiterjesztés kötelező expert módban** — egy csupasz név, mint a `.include "macros"` , `.include "macros.json"` -ként kezelődik.
- Útvonal-feloldás: először a projektfájl mellett próbálja (relatív), majd az app beépített `samples/` könyvtárára esik vissza.

Generált ASM (nincs cím-felülírás):

```
; .include "library.json" – 12 blokk
... (a kifejtett blokkok következnek)
```

Generált ASM (`C000` betöltési címmel):

```
; .include "library.json" @ $C000 – 12 blokk
*=$C000
... (a kifejtett blokkok következnek)
```

Tipp: használd az `INCLUDE`-ot újrahaználható szubrutin-könyvtárak építésére, amelyeket megoszthatsz projektek közt. Az `.inc` / `.asm` / `.s` fájlok akkor a legjobbak, ha sima szövegszerkesztőben akarod szerkeszteni a könyvtárat, vagy más 6502 assemblerekkel megosztani; a `.json` akkor, ha a könyvtár magában a Visual Assemblerben készült. Állíts be betöltési címet, ha a könyvtárnak nincs saját `ORG`-ja, vagy ha felül akarod írni az alapértelmezett elhelyezését.

TABLE

Mint a **DIM egy konkrét címen** — nevet ad egy lookup táblának, és beállítja, hol él a memóriában. Tegyél utána BYTE, WORD vagy FILL blokkokat a tábla tartalmának definiálásához.

Mező	Leírás
Name	Címke-azonosító a táblához (pl. <code>color_table</code>)
Address	Fix cím, ahol a tábla kezdődik (pl. <code>\$C000</code>)

Expert szintaxis:

```
.table color_table, $C000
```

Generált ASM:

```
color_table:
```

A program counter a megadott címre ugrik. Tegyél BYTE/WORD/FILL blokkokat a TABLE után a tartalom feltöltéséhez.

Méret: 0 byte.

ORG

Beállítja, hova kerül a program (vagy egy szakasza) a memóriában — mintha egy kezdőcímet választanál, mielőtt gépi kódot gépelnél. Minden programnak legalább egy ORG kell. A szabványos C64 BASIC-ből betölthető kezdet a `$0801` .

Mező	Leírás
Address	Az új origin cím (pl. <code>0801</code> HEX-ben, vagy <code>2049</code> DEC-ben)
HEX / DEC	A cím-bemenet váltása hexadecimális és decimális megjelenítés közt

Expert szintaxis:

```
* = $C000
```

Generált ASM:

```
* = $C000
```

Méret: 0 byte. Az ORG blokk maga nem generál gépi kódot.

Minden ORG blokk új szakaszt kezd. Az utána következő blokkok arról a címről kezdve assembleálódnak. A PRG exportjakor minden szakasz egyetlen fájlba olvad — a szakaszok közti rések nullákkal töltődnek fel.

Példa — kód a \$0801 -en, adattábla a \$C000 -en:

```
* = $0801
    LDX #$00
loop:
    LDA $C000,X
    STA $D800,X
    INX
    BNE loop
    RTS

* = $C000
    .byte $01, $02, $03, ...
```

Tipp: minden programnak ORG blokkal kell kezdődnie. C64 BASIC-ből betölthető programnál a tipikus kezdőcím a \$0801 (2049 decimális). Ha a **BASIC SYS stub** engedélyezett, az assembler egy rövid BASIC sort tesz a \$0801 -re, és a kódod a \$080D -n kezdődik.

LOOP / NEXT

Mint a **FOR X=N TO 1 STEP -1 : ... : NEXT X** BASIC-ben — N-től 1-ig számol lefelé az X vagy Y regiszterrel. Tegyél le egy LOOP blokkot, tedd az utasításaidat közte és a NEXT közé, és automatikusan a helyes számú alkalommal hurokban fut.

LOOP

Mező	Leírás
Register	X vagy Y — a számláló regiszter
Count	Ciklus-iterációk száma (hex vagy decimális, pl. 0A = 10)
Label	Automatikusan generált loop címke (pl. loop0)

Expert szintaxis:


```
.loop X, 10, loop0
```

Generált ASM:

```
LDX #$0A
loop0:
```

Méret: 2 byte (LD_ opcode + immediate operandus).

NEXT

Mező	Leírás
Register	Automatikusan a LOOP regiszteréhez illesztve
Label	Automatikusan a LOOP címkéjéhez kötve

Expert szintaxis:

```
.next loop0
```

Generált ASM:

```
DEX
BNE loop0
```

Méret: 3 byte (DEX + BNE + branch offset).

Példa — 10 képernyőcella törlése:

```
LDX #$0A
loop0:
LDA #$20      ; szökőz karakter
STA $0400,X
```

```
DEX
BNE loop0
```

FOR / ENDF

Mint a **FOR X=0 TO N-1 : ... : NEXT X** BASIC-ben — 0-tól számol *felfelé*. Ideális, amikor előre haladó indexre van szükség, pl. egy string vagy tömb végigjárásához.

FOR

Mező	Leírás
Register	X vagy Y — a számláló regiszter
Count	Ciklus-limit (hex vagy decimális, pl. \$12 = 18). Az X/Y 0-tól limit-1-ig fut.
Label	Automatikusan generált loop címke (pl. for0)

Expert szintaxis:

```
.for X, $12, for0
```

Generált ASM:

```
LDX #$00
for0:
```

Méret: 2 byte (LD_ opcode + #\$00).

ENDF

Mező	Leírás
Register	Automatikusan a FOR regiszteréhez illesztve
Label	Automatikusan a FOR címkéjéhez kötve
Count	Automatikusan a párosított FOR-ból másolva

Expert szintaxis:

```
.endf for0
```

Generált ASM:

```
INX
CPX #$12
BNE for0
```

Méret: 5 byte (IN_ + CP_ #imm + BNE offset).

Példa — null-lezárt string kiírása:

```
LDX #$00
for0:
LDA msg,X      ; msg = PETSCII string fix címen
BEQ done       ; null lezáró → kilépés
JSR $FFD2      ; CHROUT
INX
CPX #$12       ; max 18 karakter
BNE for0
done:
RTS
```

LOOP vs FOR: a LOOP lefelé számol (N→1) — jó késleltetésekhez, kitöltésekhez, pixel hurkokhoz. A FOR felfelé számol (0→N) — jó string/tömb eléréshez. Mindkettő használhat X-et vagy Y-t.

PUSH / PULL

Mint **változók mentése egy GOSUB előtt és visszaállítása utána** — de a 6502 hardveres stacket használja. Ha egy szubrutin használja A-t, X-et vagy Y-t, vedd körül PUSH-sal és PULL-lal, hogy a hívó kód regiszterei megmaradjanak.

PUSH

Egy vagy több regisztert push-ol a stackre. A sorrend mindig A → X → Y (legbelső először).

Mező	Leírás
Registers	Bármely kombináció: A , X , Y , AX , AY , XY , AXY

Expert szintaxis:

```
.push AXY
```

Generált ASM (példa: AX):

```
PHA
TXA
PHA
```

Méret: 1 byte A-hoz (PHA), 2 byte X-hez vagy Y-hoz (transfer + push).

PULL

Visszaállítja a regisztereket a stackről **fordított sorrendben** (Y → X → A).

Mező	Leírás
Registers	Ugyanaz, mint a PUSH — meg kell egyeznie a megfelelő PUSH blokkal

Expert szintaxis:

```
.pull AXY
```

Generált ASM (példa: AX):

```
PLA
TAX
PLA
```

Szabály: a PUSH és PULL mindig **azonos regiszterkészletet** kell használnjon. PUSH AX → PULL AX (belsőleg fordítva állít vissza: előbb X, majd A).

Mint az **RTS barátságosabb makró-névvel** — a `.end` egyetlen `RTS` byte-ot bocsát ki, és rövid szubrutin-lezáróként viselkedik expert módban.

Expert szintaxis:

```
.end
```

Generált ASM:

```
RTS
```

Méret: 1 byte.

Ezt akkor használd, ha egy szubrutin-vége jelölőt szeretnél, ami kicsit inkább makrónak, mint nyers utasításnak olvasható.

MACRO / ENDM / INVOKE

Mint **egy nevesített GOSUB paraméterekkel** — definiálj egyszer egy újrahasználgató kódrészletet (MACRO...ENDM), majd hívd bárhol INVOKE-kal. Adj át minden alkalommal más argumentumértékeket ahelyett, hogy blokkokat másolgatnál.

MACRO (definíció eleje)

Mező	Leírás
Name	A makró azonosítója (pl. <code>setColor</code>)
Params	Opcionális, vesszővel elválasztott paraméternevek (pl. <code>color</code> vagy <code>color, count</code>)

Jelöli a makró-definíció elejét. A MACRO és ENDM közti blokkok a makró teste — **nem generálnak kódot** ott, ahol a definíció van. Használd a `{paramName}` -t helyőrzőként az argumentumokhoz.

Generált ASM:

```
; .MACRO setColor (color)
... (test blokkok)
; .ENDM
```

Expert-mód szintaxis:

```
.macro setColor color
    LDA {color}
    STA $D020
.endm
```

ENDM (definíció vége)

Lezárja az aktuális makró-definíciót. Nincs mező.

INVOKE

Meghív egy definiált makrót ezen a pozíción, és behelyettesíti a megadott argumentumértékeket a test `{paramName}` helyőrzőibe.

Mező	Leírás
Macro name	Válaszd ki a definiált makrók legördülőjéből
Arguments	Vesszővel elválasztott argumentumértékek, amelyek illeszkednek a makró paraméterlistájához (pl. <code>#\$07</code>)

Generált ASM:

```
; .invoke setColor($07)
    LDA #$07
    STA $D020
```

Expert-mód szintaxis:

```
; egyetlen argumentum:
.invoke setColor($07)

; több argumentum:
.invoke drawPixel($10, $20)

; nincs argumentum:
.invoke clearScreen

; szöveg / string argumentumok (idézőjelben):
.invoke printText("Hello, World!")

; .call alias (a .invoke szinonimája):
.call setColor($07)
```

A makró teste inline kifejtődik, a `{paramName}` a tényleges argumentumokra cserélve. A szóközzel elválasztott forma (`.invoke setColor #$07`) is elfogadott.

Argumentum-típusok:

- **Numerikus:** `#$07`, `$10`, `255` — hex vagy decimális értékek
- **Szöveges stringek:** `"Hello, World!"` — idézőjeles stringek; az idézőjeleken belüli vesszők a szöveg részének számítanak, nem argumentum-elválasztónak
- **Vegyes:** `#$07`, `"hello"`, `$20` — bármely kombináció

Tipp: definiáld a makrókat a programod tetején (vagy alján), majd INVOKE-old őket, ahol szükséges. A makrók többször is meghívhatók különböző argumentumokkal.

REGION / ENDREGION

Tisztán vizuális csoportosítás — **nulla byte**, nulla hatás a lefordított kódra. Mint egy BASIC-program szakaszának nevesített blokká hajtása, hogy becsukhasd, és másra fókuszálhass.

Mező	Leírás
Region name	Szabad szöveges címke a szakaszhoz (pl. <code>init</code> , <code>game_loop</code> , <code>sprite_setup</code>)

Expert szintaxis:

```
.region init
    ; blokkok...
.endregion
```

Vezérlők a REGION blokk fejlécén (mindig láthatók):

-  /  **kapcsoló** — becsukja vagy kinyitja a teljes régiót. Becsukva a REGION és ENDREGION közti minden blokk el van rejtve.
-  **Expand all** — kinyit minden egyénileg becsukott blokkot a régióon belül, és szükség esetén magát a régiót is.
-  **Select in ASM** — kiemeli a teljes régió kód-tartományát az ASM nézetben (a `; ===[ name ]===`-től a `; ===[/name]===`-ig), és odagörget. Automatikusan az ASM földre vált, ha nem látható.
-  **Copy region** — a REGION blokkot, minden gyerekblokkot és a hozzá tartozó ENDREGION-t vágólapra másolja. Egy ✓ villanás megerősíti a másolást.
-  **Paste region** — a másolt régiót új régióként szúrja be közvetlenül az aktuális régió ENDREGION-je után, és odagörget. A gomb halvány, amíg nincs másolt régió.

Generált ASM:

```

; region init
    SEI
    LDA #$00
    STA $D020
; endregion init

```

Méret: 0 byte a REGION és az ENDREGION esetén is.

Példa munkafolyamat:

1. Adj hozzá egy **REGION** blokkot, a régió nevét állítsd **init**-re.
2. Add hozzá az inicializáló utasításaidat alá.
3. Adj hozzá egy **ENDREGION** blokkot a szakasz lezárásához.
4. Kattints a ►-re a REGION-on, hogy egy sorra csukd az egész szakaszt, amíg a program más részein dolgozol.

Megjegyzés: a régiók **egymásba ágyazhatók**. Minden ENDREGION a legközelebbi nyitott REGION-t zárja. Nincs hatás a lefordított kimenetre.

DEFINE / IF / ELSE / ENDIF

Mint **egy kapcsoló, amit az assembler olvas** — a **DEFINE DEBUG** bekapcsol egy szimbólumot, majd minden **IF DEBUG** blokk bekerül, és az **ELSE** ága kimarad. Vedd ki a DEFINE blokkot, és az IF blokk eltűnik a kimenetből. Release build-ekhez nem kell kódot törölni.

DEFINE

Mező	Leírás
Symbol	Egy vagy több, vesszővel elválasztott aktiválandó azonosító (pl. DEBUG vagy DEBUG, PAL)

Expert szintaxis:

```

.define DEBUG, PAL

```

Generált ASM:

```

; .DEFINE DEBUG
; .DEFINE DEBUG, PAL

```


Egy `DEFINE` blokk egyszerre több szimbólumot is aktiválhat (vesszővel elválasztva). Tedd a `DEFINE` blokkokat a programod tetejére. A blokk eltávolítása azonnal deaktiválja minden szimbólumát.

IF

Mező	Leírás
Condition	Tesztelendő azonosító (egy <code>DEFINE</code> szimbólumhoz kell illeszkednie ahhoz, hogy aktív legyen)

Expert szintaxis:

```
[  
  
    .if DEBUG  
  
]
```

Generált ASM:

```
[  
  
    ; .IF DEBUG  
  
]
```

Az `IF` és `ENDIF` (vagy `ELSE`) közti blokkok bekerülnek vagy kimaradnak attól függően, hogy a feltétel-szimbólumnak van-e illeszkedő `DEFINE`-ja a programban. A kimaradt blokkok `; [IF skipped] ...` megjegyzésként jelennek meg, és **nulla byte-ot** generálnak.

ELSE

Nincs mező. Jelöli az alternatív ágot — akkor assembleálódik, amikor az `IF` feltétel *nem* aktív.

Expert szintaxis:

```
[  
  
    .else  
  
]
```

Generált ASM:

```
[  
  
    ; .ELSE  
  
]
```

ENDIF

Nincs mező. Lezárja a feltételes blokkot.

Expert szintaxis:

```
.endif
```

Generált ASM:

```
; .ENDIF
```

Méret: 0 byte mind a négy bloknál. Csak a *köztük* lévő tartalom számít.

Példa — debug keret-villantás, a release build kihagyja:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02      ; piros keret
    STA $D020
; .ELSE
    LDA #$00      ; fekete (release)
    STA $D020
; .ENDIF
```

Példa — több szimbólum egy DEFINE blokkban:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx      ; PAL időzítési konstans
; .ELSE
    LDA #$xx      ; NTSC időzítési konstans
; .ENDIF
```

Az egymásba ágyazott **IF** blokkok támogatottak. Ha egy külső blokk kimarad, a belső blokkok is kimaradnak.

Megjegyzés: ez fordítási idejű feltételkezelés. Futásidejű compare/branch cukorért lásd a **Futásidejű IF / ELSE / ENDIF**-et lentebb.

.ASSERT

(Új a 2.3.9-ben.) Egy **fordítási idejű józansági ellenőrzés**. A `.assert` egy kifejezést értékel ki assembleálás közben; ha hamis (`0`), a build egy beszédes hibával leáll, amely tartalmazza a tényleges értéket. Ha igaz (nem nulla), semmit nem bocsát ki.

Mező	Leírás
Expression	Bármely assembler-kifejezés: címkék, <code>CONST</code> -ok, <code>*</code> (program counter), aritmetika és összehasonlítások (<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>)
Message	Opcionális szöveg, amely a hibaüzenethez fűződik

Expert szintaxis:

```
.assert spriteData < $C000
.assert * < $A000
.assert end - start <= 256, "a sprite tábla túllépett egy lapot"
```

Viselkedés:

- **Méret:** 0 byte.
- Egy hamis assertion megszakítja az assembleálást:
`` .assert end - start <= 256`` hamis (érték: `0`). a sprite tábla túllépett egy lapot
- Az a kifejezés is hibára fut, amelyet nem lehet kiértékelni (nem definiált címke, stb.),
„nem értékelhető ki fordítási időben” üzenettel.
- Az összehasonlítások `1 / 0`-t adnak; tedd a `.assert`-et bárhova a program folyamában — azon a címen ellenőrződik, ahol áll, így a `.assert * < $D000` az aktuális kimeneti pozíciót teszteli.

Példa — sprite blokk védeke lap-átlépés ellen:

```
* = $3000
sprite_data:
    .byte $00, $00, $00 ; ... 63 byte ...
sprite_data_end:
    .assert sprite_data_end - sprite_data == 63, "a sprite pontosan 63 byte kell legyen"
    .assert (sprite_data % 64) == 0, "a sprite 64 byte-ra igazított kell legyen"
```

CONST

Mint **egy nevesített változó, amely sosem változik** — `SCREEN = $0400`. Használd a nevet ahelyett, hogy mindenhol nyers címeket gépelnél, így a kód olvashatóbb és később könnyebben módosítható.

Mező	Leírás
Name	A konstans azonosítója (pl. <code>SCREEN</code>)
Value	Numerikus érték a választott számrendszerben (pl. <code>0400</code> HEX-ben = <code>\$0400</code> cím), vagy egy PC-relatív kifejezés (lásd lentebb)
Format	HEX vagy DEC — vezérli, hogyan adod meg és jelenik meg az érték

Expert szintaxis:

```
.const SCREEN = $0400
.const FRAMES_1S = 60
```

Generált ASM:

```
; .CONST SCREEN = $0400
```

A konstans neve megjelenik a **címke-választó** legördülőben az utasításblokkokon — csak kattints rá a beszúráshoz.

PC-relatív kifejezések (`*+N` / `*-N`):

Az érték-mező elfogad `*+N` -t vagy `*-N` -t is, ahol a `*` magának a CONST blokknak a fordítási címe. Ezzel egy nevesített alias hozhatsz létre egy közeli utasításon belüli byte-hoz — a klasszikus önmódosító kód minta:

```
CONST op = *+1      ; op → a következő LDA immediate operandusának címe
LDA #$00            ; a $00-t futásidőben módosítjuk
...
STA op              ; felülírja a #$00 byte-ot → az LDA legközelebb az új értéket olvassa
```

A CONST 0 byte-ot bocsát ki; a címke fordítási időben `aktuális_cím + 1` -re oldódik.

Aritmetikai kifejezések:

Az érték-mező elfogad általános aritmetikát, beleértve a korábban definiált CONST nevekre való hivatkozásokat, hex/bináris literálokat és beépített matematikai függvényeket:

```
.const SCREEN      = $0400
.const SCREEN_END  = SCREEN + 40*25 ; 1000 byte-tal később
.const COLOR_RAM   = $D800
```

```
.const MID_X      = 160
.const SIN_TABLE  = round(sin(PI/8) * 127) ; előre kiszámolt szinuszárték
```

Beépített függvények: `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, `PI` konstans

Operátorok: `+` `-` `*` `/` Literálok: `$FF` (hex), `%10110000` (bináris) Alsó/felső byte: `lo(expr)`, `hi(expr)`

Méret: 0 byte.

VAR

Mint a **CONST**, **de automatikusan foglalva** — a `VAR` zero-page tárhelyet foglal egy címkéhez anélkül, hogy be kellene gépelned a címet. Használd számlálókhoz, pointerekhez és rövid életű, ZP-be való állapothoz.

Mező	Leírás
Name	Változónév / címke
Size (opcionális)	Foglalandó byte-ok száma. Hagyd el egy byte-hoz.

Expert szintaxis:

```
.var counter
.var timer, 2
.var lives
```

Gyakorlati példa:

```
.region Vars
.var counter
.var timer, 2
.endregion

LDA #$00
STA counter
```

Generált ASM:

```
; .var counter
```

Méret: alaphól 1 byte, vagy N byte, ha megadod a méretet.

Az allokátor egy konfigurálható zero-page kurzort jár be (\$02 -tól \$FE -ig), és a következő szabad slotot rendeli hozzá. Ha a kért terület átfed egy már használt címkével, a fordító figyelmeztetést ad.

Futásidejű IF / ELSE / ENDIF

Mint **egy valódi elágazás-sablon** — ez a változat futásidőben működik, nem fordítási időben. Összehasonlítja A, X vagy Y értékét egy immediate értékkel, és a helyes CMP / CPX / CPY + elágazás szekvenciát bocsátja ki neked.

Mező	Leírás
Register	A, X vagy Y
Operator	==, !=, <, <=, >, >=
Value	Immediate érték HEX vagy DEC formátumban

Expert szintaxis:

```
.if A == #$10
    LDA #$07
.else
    LDA #$0F
.endif
```

Méret: a választott elágazásoktól és az összehasonlítási formától függ.

Az összehasonlítás alaphól előjel nélküli. A <= és > esetén a makró a kiválasztott regiszterhez a legrövidebb ekvivalens elágazás-láncra fejt ki.

WHILE / ENDW

Mint **egy futásidejű hurok tesztel a tetején** — a test addig fut, amíg a feltétel igaz marad.

Mező	Leírás
Register	A, X vagy Y
Operator	==, !=, <, <=, >, >=
Value	Immediate érték HEX vagy DEC formátumban

Expert szintaxis:

```

    .while A != #$00
        JSR getchar
    .endw

```

Méret: a hurok testétől és az összehasonlítási formától függ.

Használd a **WHILE** -t, ha a hurok az első iteráció befejezése előtt is véget érhet. Ez a szám-alapú **LOOP / NEXT** segéd futásidejű megfelelője.

REPEAT / UNTIL

Mint **egy futásidejű hurok tesztel az alján** — a test mindig lefut legalább egyszer, majd a feltétel dönt a leállásról.

Mező	Leírás
Register	A , X vagy Y
Operator	== , != , < , <= , > , >=
Value	Immediate érték HEX vagy DEC formátumban

Expert szintaxis:

```

.repeat
    JSR getchar
.until A == #$00

```

Méret: a hurok testétől és az összehasonlítási formától függ.

Használd a **REPEAT / UNTIL** -t, ha azt akard, hogy a test legalább egyszer lefusson a kilépési ellenőrzés előtt.

MEMCPY / MEMSET

Mint **kis memória-rutinok, amelyekhez folyton nyúlsz** — a **MEMCPY** egy összefüggő blokkot másol, a **MEMSET** egy tartományt tölt fel egy byte-tal.

Makró	Mezők
MEMCPY	src , dst , size
MEMSET	addr , value , size

Expert szintaxis:

```
.memcpy src=$C000, dst=$D000, size=$0100
.memset addr=$0400, value=#$20, size=$03E8
```

Generált ASM: inline másoló/kitöltő hurkok, a kért mérethez választva.

A 256 byte-ig terjedő méretek rövid 8 bites hurkot használnak. A nagyobb méretek automatikusan 16 bites számlálóra váltanak.

PRINT / PRINT_CHAR / PRINT_HEX / CLEAR_SCREEN / WAIT_KEY / DELAY / SET_BORDER / SET_BG

PRINT

Mint **PETSCII kimenet a sablonkód nélkül** — egy stringet ír ki `CHROUT`-on át ugyanazzal a kis/nagybetű kezeléssel, mint a PETSCII blokk. A kisbetűs jelölőnégyzet közös a PETSCII kódolóval, így a szöveg-útvonal konzisztens marad.

Expert szintaxis:

```
.print "HELLO"
.print "hello", lower
```

PRINT_CHAR

Egyetlen PETSCII byte-ot ír ki numerikus kód alapján, `CHROUT`-on átküldve. Az érték lehet nevesített const vagy címke is, amely assembleáláskor byte-ra oldódik — Blokk és Expert módban egyaránt.

Expert szintaxis:

```
.print_char 65
.print_char $41
.print_char color
```

PRINT_HEX

Egy 8 bites értéket ír ki hexadecimális szöveggként a szokásos KERNAL kimeneti útvonalon.

Expert szintaxis:


```
.print_hex A
```

CLEAR_SCREEN

Rövidítés a szabványos C64 képernyőtörlés vezérlőkódhoz.

Expert szintaxis:

```
.clear_screen
```

WAIT_KEY

Vár, amíg egy billentyűt le nem nyomnak, így nem kell minden alkalommal kézzel megírni a `GETIN` hurkot.

Expert szintaxis:

```
.wait_key
```

DELAY

A kért számú frame-et várja egy közös segédrutinon keresztül. Ezt használd rövid szünetekhez és időzítési résekhez, amikor egy teljes egyéni hurok túlzás lenne. A frame-szám lehet nyers szám vagy nevesített const, és a `.wait` csak a `.delay` aliasa.

Expert szintaxis:

```
.delay 29  
.wait 29  
.delay frames=FRAMES_1S
```

Blokk módban a delay mező kompakt const-választót használ, ha van szimbolikus érték, így nem kell minden alkalommal kézzel begépelni a nevet.

SET_BORDER / SET_BG

Kényelmi burkolók a VIC-II szín-regiszterekhez. A szín-érték lehet nyers szám vagy nevesített const, amely 0–15-re oldódik. Blokk és Expert mód is elfogad szimbolikus const-neveket itt.

Expert szintaxis:

```
.set_border 6
.set_bg 0
.set_border color
.set_bg color
```

Méret: minden segéd egy kicsi regiszter-írási szekvenciára vagy rövid KERNAL hívásra fejt ki.

Blokk módban ezek a mezők is a const-választót használják, így a szimbolikus érték látható marad ahelyett, hogy nyers számra cserélődne.

IRQ_SETUP

Egy lépésben állít be egy raszter IRQ kezelőt. A makró beírja az IRQ vektort, engedélyezi a raszter IRQ-kat, beállítja a sort, letiltja a gyakori CIA IRQ forrásokat, és CLI -vel tér vissza a normál végrehajtáshoz.

Mező	Leírás
Handler	IRQ rutin címkéje (pl. <code>my_irq</code>)
Raster	Raszter sor hex vagy decimális formában (pl. <code>\$FA</code>)

Expert szintaxis:

```
.irq_setup handler=my_irq, raster=$FA
```

Méret: egy kis beállító szekvencia; a pontos hossz a választott raszter sortól függ.

Ezt akkor használd, ha a gyakori „SEI / kezelő beírása / IRQ engedélyezés / CLI” sablonkódot szeretnéd anélkül, hogy szétszórná a programban.

RAND

Mint **egy kicsi beépített PRNG** — 8 bites pszeudo-véletlen értéket ad egy kompakt zero-page seedből.

Mező	Leírás
Seed	Opcionális zero-page seed byte vagy címke (alap <code>\$FB</code>)

Expert szintaxis:

```

    .rand

```

Méret: néhány byte, a választott implementációs úttól függően.

A generátor játékmenetre, effekt-variációra és gyors tesztadatra való. Szándékosan kicsi, nem kriptográfiailag kifinomult.

SPRITE_INIT

Egy blokkban állít be egy VIC-II sprite-ot — ahelyett, hogy ~6 POKE utasítást írnál BASIC-ben, csak töltsd ki a mezőket. Beállítja a sprite adat-pointerét, bekapcsolja, opcionálisan engedélyezi a multicolor módot, és beállítja a színét.

Mező	Leírás
Sprite #	Sprite szám 0–7
Colour	Színindex 0–15 (C64 paletta)
Data page	Sprite adat cím / 64 (pl. \$21 , ha az adat a \$0840 -en van)
Multicolor	A sprite multicolor bitjét kapcsolja (\$D01C)

Expert szintaxis:

```

.sprite_init 0, 7, $21
.sprite_init 0, 7, $21, multicolor
.sprite_init 0, 7, $21, mono

```

Generált ASM:

```

    LDA #$21
    STA $07F8      ; sprite pointer regiszter ($07F8 + N)
    LDA $D015
    ORA #$01      ; sprite 0 engedély bit beállítása
    STA $D015
    LDA $D01C
    AND #$FE      ; sprite 0 multicolor bit törlése
    STA $D01C
    LDA #$07
    STA $D027      ; sprite 0 szín-regiszter

```

Méret: 26 byte.

Sprite data page: `data_address ÷ 64` . Az alapértelmezett BASIC SYS stubbal a `JMP main` utáni `ALIGN 64` a sprite adatot a `$0840` -re helyezi → `page = $21` .

SPRITE_POS

Mint a `POKE 53248, x : POKE 53249, y` BASIC-ben — beállítja egy sprite kezdőpozícióját. A koordináták assembleáláskor be vannak égetve, és a mezők elfogadnak const-okat Blokk módban; animációhoz használj `INC` / `DEC` -et közvetlenül a sprite regiszteren.

Mező	Leírás
Sprite #	Sprite szám 0–7
X	Vízszintes pozíció 0–319
Y	Függőleges pozíció 0–255

Expert szintaxis:

```
.sprite_pos 0, 152, 100
```

Generált ASM (példa: sprite 0, X=152, Y=100):

```
LDA #$98      ; X alsó byte
STA $D000     ; sprite 0 X regiszter
LDA $D010
AND #$FE      ; sprite 0 X MSB törlése (X ≤ 255)
STA $D010
LDA #$64      ; Y = 100
STA $D001     ; sprite 0 Y regiszter
```

X > 255 esetén a makró a `$D010` megfelelő bitjét beállítja ahelyett, hogy törölné.

Méret: 18 byte.

Megjegyzés: a `SPRITE_POS` beégeti az X/Y-t a kódba (`LDA #$xx`). Sprite futásidejű animálásához használj `INC $D000` / `DEC $D000` -t — lásd a `sprite-macro-demo` mintát.

WAIT_RASTER

Megvárja, míg a VIC-II elektronsugár elér egy adott scanline-t — mint a TV frame-re szinkronizálás. Tedd ezt a game hurkod tetejére a sprite tearing megelőzéséhez. Nincs JSR, nem kell címke.

Mező	Leírás
Raster line	Cél raszter sor hexben (pl. FF = 255. sor)

Expert szintaxis:

```
.wait_raster $FF
```

Generált ASM:

```
wait:
    LDA $D012      ; aktuális raszter sor
    CMP #$FF       ; cél sor
    BNE wait       ; vissza a hurokba (-7 byte)
```

Méret: 7 byte (a BNE offset \$F9 = -7 mindig az LDA -ra mutat vissza).

Tipp: tedd a WAIT_RASTER -t a game hurkod tetejére a kijelzővel való szinkronizáláshoz és a sprite tearing megelőzéséhez.

JOYSTICK

Mint a PEEK(\$DC00) olvasása, majd a sprite pozíció POKE-olása — de egy blokkban. Beolvassa az egyik CIA joystick portot, és ennek megfelelően állítja egy sprite X/Y regisztereit. Teljesen inline, nem kell JSR.

Mező	Leírás
Port	1 = port 1 (\$DC01) vagy 2 = port 2 (\$DC00)
Sprite #	Sprite szám 0–7 (melyik X/Y regiszterpár frissül)

Expert szintaxis:

```
.joystick 2, 0
```

Generált ASM (port 2, sprite 0):

```

    LDA $DC00      ; CIA port 2 olvasása
    LSR            ; 0. bit → carry (Fel)
    BCS skip_up    ; carry beállítva = NINCS nyomva
    DEC $D001      ; Y-1 (fel mozgatus)
skip_up:
    LSR            ; 1. bit → carry (Le)
    BCS skip_down
    INC $D001      ; Y+1 (le mozgatus)
skip_down:
    LSR            ; 2. bit → carry (Balra)
    BCS skip_left
    DEC $D000      ; X-1 (balra mozgatus)
skip_left:
    LSR            ; 3. bit → carry (Jobbra)
    BCS skip_right
    INC $D000      ; X+1 (jobbra mozgatus)
skip_right:
```

Joystick bittérkép (active-LOW — bit = 0 jelenti a nyomást):

Bit	Írány	CIA regiszter
0	Fel	\$DC00 (port 2) / \$DC01 (port 1)
1	Le	
2	Balra	
3	Jobbra	
4	Tűz	(ez a makró nem kezeli)

Méret: 27 byte. A BCS offset mindig +3 (átugorja a következő 3 byte-os DEC / INC abs utasítást).

Tipikus használat: tedd egy gameloop címke belsejébe, elől WAIT_RASTER -rel:

```

gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    JMP gameloop
```

Beolvas egy Commodore 1351 arányos egeret, és mozgat egy sprite-ot. Teljesen **inline** — nem kell JSR vagy címke. A makró kiválasztja a CIA portot, megvárja, míg a SID paddle bemenetek beállnak, majd dekódolja a delta mozgást a szabványos 1351 driver mintával, és alkalmazza a sprite regiszterekre.

Mező	Leírás
Port	1 = CIA \$DC00 7:6 bitek = %01 ; 2 = %10
Sprite #	Sprite szám 0–7
ZP byte X	Zero-page cím (hex) az előző POTX minta tárolására (pl. FD)
ZP byte Y	Zero-page cím (hex) az előző POTY minta tárolására (pl. FE)

Generált ASM forma (port 1, sprite 0, ZP \$FD / \$FE):

```

; CIA port kiválasztás + beállítás
LDA $DC00
AND #$3F
ORA #$40
STA $DC00
LDX #$67
wait:
DEX
BNE wait

; X tengely – szabványos 1351-stílusú 7 bites delta dekódolás
LDA $D419
TAY
SEC
SBC $FD
AND #$7F
LDX #$00
CMP #$40
BCS xneg
LSR A
BEQ xdone
STY $FD
CLC
ADC $D000
STA $D000
TXA
ADC #$00
AND #$01
BEQ xdone
LDA $D010
EOR #$01
STA $D010
JMP xdone
xneg:
ORA #$C0
CMP #$FF
BEQ xdone
```

```

SEC
ROR
DEX
STY $FD
CLC
ADC $D000
STA $D000
TXA
ADC #$00
AND #$01
BEQ xdone
LDA $D010
EOR #$01
STA $D010
xdone:

; Y tengely – ugyanaz a dekódolás, majd invertálás alkalmazás előtt
LDA $D41A
TAX
SEC
SBC $FE
AND #$7F
CMP #$40
BCS yneg
LSR A
BEQ ydone
STX $FE
EOR #$FF
SEC
ADC $D001
STA $D001
JMP ydone
yneg:
ORA #$C0
CMP #$FF
BEQ ydone
SEC
ROR
STX $FE
EOR #$FF
SEC
ADC $D001
STA $D001
ydone:

```

Méret: 142 byte.

Expert-mód szintaxis:


```
.mouse port, spriteNum, zpX, zpY
; példa:
.mouse 2, 0, FD, FE
```

Fontos: az első hívás előtt inicializáld a zero-page byte-okat az aktuális POTX/POTY értékekkel, hogy elkerüld az ugrást az első frame-en:

```
; port 1: LDA $DC00 : AND #$3F : ORA #$40 : STA $DC00
; port 2: LDA $DC00 : AND #$3F : ORA #$80 : STA $DC00
LDA $D419 : LSR A : AND #$3F : STA $FD
LDA $D41A : LSR A : AND #$3F : STA $FE
```

Tipp: olvasd az egeret frame-enként egyszer — tedd a `WAIT_RASTER` -t a game hurokban a `MOUSE` elé.

SPRITE_COL

Mint a `PEEK($D01E)` BASIC-ben — ellenőrzi a VIC-II hardveres ütközés-regisztereket, és megmondja, hogy egy sprite eltalált-e egy másik sprite-ot vagy a háttérrel. Teljesen inline, nem kell JSR.

Mező	Leírás
Sprite #	Sprite szám 0–7 (melyik sprite bitjét ellenőrizze)
Collision type	<code>Sprite-Sprite (\$D01E)</code> — ütközés másik sprite-tal; <code>Sprite-Background (\$D01F)</code> — ütközés a háttér-grafikával

Expert szintaxis:

```
.sprite_col 0, sprite
.sprite_col 0, background
```

Generált ASM (sprite 0, sprite–sprite):

```
LDA $D01E      ; sprite-sprite ütközés-regiszter olvasása (törli is!)
AND #$01       ; 0. bit izolálása (sprite 0)
```

; A ≠ 0 → ütközés történt

Méret: 5 byte.

Fontos: a `$D01E` / `$D01F` olvasása **törli a regisztert**. Olvasd frame-enként egyszer, és azonnal reagálj az eredményre `BEQ` / `BNE` -vel.

Tipikus használat:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    SPRITE_COL (sprite=0, type=sprite-sprite)
    BEQ no_hit          ; A = 0 → nincs ütközés
    LDA #$02
    STA $D020           ; piros keret = találat!
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; világoskék keret = tiszta
    JMP gameloop
```

Lásd még: `collision-demo` minta — zöld labda (sprite #0) vs. piros kereszt (sprite #1).

LOADFILE

Mint a `LOAD "file",8` BASIC-ben — betölt egy fájlt egy D64 lemezről futásidőben a KERNAL LOAD rutinjával. Ezzel tölthetsz adatot, zenét vagy extra kódot lemezről, miközben a programod fut.

Mező	Leírás
Filename	Fájlnév a lemezen (max 16 karakter, automatikus nagybetűsítés; a <code>,</code> , <code>"</code> , <code>/</code> , <code>\</code> , <code>:</code> , <code>*</code> , <code>?</code> , <code><</code> , <code>></code> , <code> </code> karakterek kiszűrődnek)
Device	Eszközsám 8–30 (alap <code>8</code>)
Override address (opcionális)	Hex betöltési cím (pl. <code>C000</code>). Ha beállított, a fájl erre a címre töltődik (<code>sec=0</code> , a PRG fejléct figyelmen kívül hagyva). Hagyd üresen a fájl saját 2 byte-os PRG fejlécének használatához (<code>sec=1</code>).
Error label (opcionális)	Ha beállított, egy <code>BCS</code> utasítás generálódik a JSR LOAD után. Ha a KERNAL carry beállítással (hiba) tér vissza, a végrehajtás erre a címkére ugrik.

Expert szintaxis:

```
.loadfile "DEMO-COLORS", 8
.loadfile "DEMO-COLORS", 8, $C000
.loadfile "DEMO-COLORS", 8, $C000, error_label
```

Generált kód szerkezete:

```
JMP skip_filename      ; átugrik az inline fájlneven
.byte "DEMO-COLORS"    ; fájlnev byte-ok (PETSCII, 11 karakter)
skip_filename:
LDA #11                ; fájlnev hossza
LDX #<fname            ; pointer lo
LDY #>fname            ; pointer hi
JSR $FFBD              ; SETNAM
LDA #1                 ; logikai fájl #1
LDX #8                 ; device 8
LDY #0                 ; sec=0 (override cím) vagy #1 (fájl saját címe)
JSR $FFBA              ; SETLFS
LDA #0                 ; LOAD parancs (nem VERIFY)
JSR $FFD5              ; LOAD
BCS fail               ; (csak ha van error label)
```

Méret: $3 + \text{fájlnev_hossz} + 9 \text{ (SETNAM)} + 9 \text{ (SETLFS)} + (4 \text{ ha override}) + 5 \text{ (LOAD)} + (2 \text{ ha error label})$ byte. Minimum 27 byte.

Fontos: a fájlnev inline tárolódik a gépi kódban közvetlenül egy `JMP skip_filename` után. A lemezen lévő fájlnevnek nagybetűs PETSCII-nek kell lennie — ami megegyezik a sima ASCII nagybetűkkel (A – Z). A makró ezt automatikusan kikényszeríti.

Éles programokhoz mindig használj error label-t — ha a fájl nincs meg, a KERNAL beállítja a carry flag-et, és a végrehajtás átcúszik arra, ami utána van.

Lásd még: `loadfile-demo` minta — `DEMO-COLORS.PRG` betöltése D64-ről BCS hibaágyal és vizuális hibaképernyővel.

EXODECRUNCH

Programon belüli **Exomizer kitömörítés**. Ezt a makrót közvetlenül egy olyan `LOADFILE` után használd, amely egy Exomizer `mem`-módú tömörített streamet töltött be — az EXODECRUNCH visszafelé csomagolja ki a streambe ágyazott címre.

Mező	Leírás
Depacker address	Ahol a depacker kód él a memóriában (alap <code>B000</code>). 16 bites hex cím kell legyen.

Expert szintaxis:

```
.exodecrunch
.exodecrunch depacker=$B000
```

Generált kód (19 byte):

```
LDA $AE          ; KERNAL load-end lo (az előző LOAD állította be)
STA $04          ; depacker src-end pointer lo
LDA $AF          ; KERNAL load-end hi
STA $05          ; depacker src-end pointer hi
LDA #$36         ; BASIC ROM ki ($A000-$BFFF RAM lesz)
STA $01
JSR depacker      ; a depacker hátulról előre olvas a $04/$05-ön át
LDA #$37         ; alapértelmezett mapping visszaállítása (BASIC + KERNAL + I/O)
STA $01
```

Hogyan működik:

1. A KERNAL `LOAD` (\$FFD5) a ZP `$AE/$AF` -t az utoljára betöltött byte utáni pozícióra állítja. Az EXODECRUNCH ezt a ZP `$04/$05` -re másolja, ami a hivatalos Exomizer visszafelé-forrás-vég konvenció.
2. A depacker jellemzően a `$B000` -en van (a BASIC ROM-ra mappelt tartományon belül). A makró `$01 = $36` -ra kapcsol, hogy a CPU ott RAM-ot lásson a JSR alatt, majd utána visszaállítja `$01 = $37` -re.
3. A kitömörítési célcím **magába a tömörített streambe van kódolva**, amikor `exomizer mem -l <load> file, <target>` -rel tömörítesz — a depacker a stream első byte-jaiból olvassa ki.

Depacker bináris: az előre elkészített visszafelé-depacker a `samples/exo-decrunch.bin` (477 byte, ORG \$B000). Ez a hivatalos `exodecrunch.asm` Kick Assembler-es burkolása, minden olvasáshoz hozzáadott `INC $D020` -szal, hogy a kitömörítés alatt látható keret-villantás legyen. Tedd a programodba egy `INCBIN` blokkal a depacker címén.

Safety offset kompenzáció: az Exomizer alapértelmezett mem módja 2 byte-os safety offsetet alkalmaz — az adat 2 byte-tal korábbra kerül a kért célnál. A Run via D64 dialógus **automatikusan hozzáad 2-t a Dst mezőhöz** az exomizer hívása előtt, így a látható viselkedés a beírt címhez igazodik.

Lásd még: `exo-multicolor-demo` minta — teljes végponttól végpontig példa: `LOADFILE`-l al egy tömörített multicolor bitmap a `$C000`-re, `EXODECRUNCH` kicsomagolja a `$2000`-re, majd `screen` → `$0400` és `color` → `$D800` másolás, és a VIC-II multicolor bitmap módra váltás.

Integrációs teszt: `cargo test --test exomizer_integration` (a `src-tauri/` -ban) ellenőrzi a teljes tömörítés + kitömörítés round-tripet egy 6502 emulátoron a valódi depacker binárisal. Elfogadási kritérium: 10000 byte byte-azonos a forrás `multi-color.bin` -nel.

REU_CHECK

Detektálja, hogy be van-e dugva egy Commodore RAM Expansion Unit (REU) — mint a `PEEK($D010)` ellenőrzése, hogy jelen van-e a hardver. Két minta beírásával és visszaolvasásával tesztel a REU `$DF04` regiszterén.

Mező	Leírás
Nincs	Nincs operandus

Generált kód (34 byte):

```
LDA #$55
STA $DF04
LDA $DF04
CMP #$55
BNE fail
LDA #$AA
STA $DF04
LDA $DF04
CMP #$AA
BNE fail
LDA #$00
CMP #$FF ; Z=0 => REU jelen van
BNE done
fail:
LDA #$FF
CMP #$FF ; Z=1 => nincs REU
done:
```

A makró normalizálja az eredményt, hogy a következő elágazás egyszerű maradjon: `BNE` = REU jelen van, `BEQ` = REU hiányzik.

Expert szintaxis:

```
.reu_check
```

Eredmény a flag-ekben:

- **Z = 0** (eredmény ≠ 0) → REU jelen van → használj `BNE` -t
- **Z = 1** (eredmény = 0) → nincs REU → használj `BEQ` -t

Nincs konfigurálható mező — a makró minden alkalommal ugyanazt a kódot generálja.

Tipikus használat:

```
REU_CHECK
BEQ no_reu      ; kihagyás, ha nincs REU
; ... REU kód itt ...
no_reu:
```

REU_STASH / REU_FETCH / REU_SWAP

DMA blokk-átvitel a C64 RAM és a REU bővítő memória közt — mint egy nagyon gyors POKE hurok, de a CPU nem dolgozik (a REU chip másolja az adatot, míg a CPU meg van állítva). Egy \$1000 byte-os átvitel gyakorlatilag azonnali.

Makró	Irány	\$DF01 parancs
REU_STASH	C64 RAM → REU	\$90
REU_FETCH	REU → C64 RAM	\$91
REU_SWAP	C64 RAM ↔ REU	\$92

Mezők:

Mező	Leírás	Példa
C64 address	Forrás/cél a C64 RAM-ban (hex)	C000
REU address	Forrás/cél a REU-ban (hex, 16 bites)	0000
REU bank	REU memória-bank (0–7)	0
Length	Átviendő byte-ok száma (hex, 16 bites)	1000

Expert szintaxis:

```
.reu_stash $C000, $0000, 0, $1000
.reu_fetch $C000, $0000, 0, $1000
.reu_swap $C000, $0000, 0, $1000
```

Generált kód (40 byte):

```
LDA #c64lo    STA $DF02    ; C64 cím LO
LDA #c64hi    STA $DF03    ; C64 cím HI
LDA #reuLo    STA $DF04    ; REU cím LO
LDA #reuHi    STA $DF05    ; REU cím HI
LDA #bank     STA $DF06    ; REU bank
```

```

LDA #lenLo    STA $DF07    ; hossz LO
LDA #lenHi    STA $DF08    ; hossz HI
LDA #$00      STA $DF09    ; address control (fix)
LDA #$00      STA $DF0A    ; interrupt mask (fix)
LDA #cmd      STA $DF01    ; DMA végrehajtás ($90/$91/$92 = stash/fetch/swap, azonnali)

```

Megjegyzés: a parancsok `$90/$91/$92` -t használnak (4. bit be = azonnali DMA mód). A `$DF01` -be írás elindítja az átvitelt; a CPU folytatódik, amikor befejeződik.

TURBO_SET

Beállítja az **Ultimate-64 (U64) CPU sebességet** a `$D031` regiszteren keresztül. Valódi C64-en vagy más emulátoron nincs hatása.

Mezők:

Mező	Leírás	Tartomány
Speed	CPU sebesség-index	0 = 1 MHz ... 7 ≈ 10 MHz ... 15 ≈ 48 MHz
Badline	Badline emuláció	Engedélyezve (C64 kompatibilis) / Letiltva (turbo)

A sebesség-byte így számolódik: `(speedIndex & 0x0F) | (badline_letiltva ? 0x80 : 0x00)`.

Generált kód (5 byte):

```

A9 xx    LDA #speed_byte
8D 31 D0  STA $D031

```

Expert-mód szintaxis:

```

.turbo_set 7,0    ; speed=7 (~10 MHz), badline engedélyezve
.turbo_set 15,1   ; speed=15 (~48 MHz), badline letiltva

```

Megjegyzés: ez a makró csak U64 hardvert érint. Valódi C64-en vagy más emulátoron a `$D031` -be ír, ami hatással lehet a CIA-ra, vagy figyelmen kívül maradhat.

SUPERCPU_DETECT

Ellenőrzi, hogy telepítve van-e egy **CMD SuperCPU** gyorsító — mint a `PEEK($D0B8)` , hogy `$FF` -től eltérő értéket ad-e vissza.

Generált kód (5 byte):

```
AD B8 D0    LDA $D0B8
C9 FF      CMP #$FF
```

Eredmény a flag-ekben:

- **Z = 0** → SuperCPU jelen van → használj `BNE` -t
- **Z = 1** → SuperCPU nincs → használj `BEQ` -t

Nincs konfigurálható mező.

Expert szintaxis:

```
.supercpu_detect
```

Tipikus használat:

```
SUPERCPU_DETECT
BEQ no_scpu      ; kihagyás, ha nincs SuperCPU
; ... SuperCPU turbo kód itt ...
no_scpu:
```

TURBO_ENABLE

Be- vagy kikapcsolja a **CMD SuperCPU turbo módot**. Előbb hívj `SUPERCPU_DETECT` -et, és hagyd ki ezt, ha a SuperCPU nincs jelen.

Mód	Regiszter	Hatás
Enable	<code>\$D07A</code>	Turbo bekapcsolása (SuperCPU-val akár 20 MHz)
Disable	<code>\$D07B</code>	Vissza az 1 MHz-es kompatibilitási módba

Generált kód (5 byte):


```
A9 00      LDA #$00
8D 7A D0    STA $D07A    ; (vagy $D07B a letiltáshoz)
```

Expert-mód szintaxis:

```
.turbo_enable on
.turbo_enable off
```

Megjegyzés: előbb hívj `SUPERCPU_DETECT` -et, és ágazz el a makró körül, ha a SuperCPU nincs jelen.

MAP_COPY

Egy tilemap-et másol egy forráscímről a screen RAM-ba (és opcionálisan a color RAM-ba) `LDA abs,X / STA abs,X` hurkok sorozatával. Hurok-iterációként egy 256 byte-os lap másolódik; a végén egy részleges lap `CPX #rem / BNE` -vel áll le. Nem kell JSR — minden kód inline generálódik.

Mező	Leírás
Source addr (screen)	Hex cím, ahol a map adat betöltés után él (pl. <code>C000</code>)
Screen RAM dest	Hová másolja a screen code-okat (pl. <code>0400</code>)
Size (bytes)	Összes másolandó byte — jellemzően <code>\$03E8</code> = 1000 (40×25 karakter)
Combined .bin	Bepipálva screen code-okat vár, amelyeket közvetlenül a color adat követ a <code>source + size</code> címen; a color adatot egy második menetben a Color RAM dest -re másolja
Color RAM dest	A color adat célja — alap <code>D800</code> (C64 color RAM)

Generált ASM (1000 byte-os map, csak screen):

```
LDX #$00
LDA $C000,X    ; 0. lap
STA $0400,X
INX
BNE *-9        ; hurok, míg X vissza nem fordul 0-ra (256 iteráció)
LDA $C100,X    ; 1. lap
STA $0500,X
INX
BNE *-9
```

```

LDA $C200,X    ; 2. lap
STA $0600,X
INX
BNE *-9
LDA $C300,X    ; maradék – 232 byte
STA $0700,X
INX
CPX #$E8
BNE *-11

```

Méret: $2 \text{ (LDX)} + \text{fullPages} \times 9 + (\text{rem} > 0 ? 11 : 0)$ byte szakaszonként. A combined mód megduplázza (screen szakasz + azonos color szakasz).

Párosítás a Map Editorral:

A Map Editor **Files** → **Save map + color RAM (.bin)** menüpontja egyetlen binárist exportál, ahol az első `size` byte screen code, a következő `size` byte pedig color RAM érték. Használd a MAP_COPY-t **Combined .bin** bepipálva, és a **Source addr**-t oda mutasd, ahova ez a fájl betöltődik (pl. INCBIN-nel a `$C000` -en):

```

* = $C000
    INCBIN "map-color.bin" @ $C000    ; screen code-ok $C000-$C3E7, color $C3E8-$C7CF
* = $0801
    ; ...
    MAP_COPY src=$C000 dst=$0400 size=1000 combined color_dst=$D800

```

Expert-mód szintaxis:

```

.map_copy $C000, $0400, 1000          ; csak screen
.map_copy $C000, $0400, 1000, auto, $D800 ; combined (color a src+size címen)
.map_copy $C000, $0400, 1000, $C3E8, $D800 ; explicit color forráscím

```

MAP_COPY16X16

Egy 16×16 karakteres területet másol egy kompakt 256 byte-os screen-code blokkból plusz egy hozzá tartozó 256 byte-os Color RAM blokkból. Charset Canvas exportokhoz és kis tile/kép darabokhoz való, ahol tizenhat különálló MAP_COPY sor zajos lenne.

Alapértelmezett elrendezés:

Adat	Alapértelmezett cím
16×16 screen code-ok	Forráscím (src)
16×16 color értékek	src + 256
Screen RAM cél	\$0400 + row×40 + col
Color RAM cél	\$D800 + row×40 + col

Expert-mód szintaxis:

```
.map_copy16x16 $3000, 12, 4
.map_copy16x16 $3000, 12, 4, $0400, $3100, $D800
```

A rövid forma a screen byte-okat a `$3000` -ről, a color byte-okat a `$3100` -ről másolja, és a 16×16 blokkot a 12. oszlop, 4. sorba helyezi. Az érvényes bal-felső pozíciók `col = 0..24` és `row = 0..9`, így a teljes 16×16 terület a 40×25-ös C64 szövegképernyőn marad.

Generált viselkedés:

- Tizenhat inline sor-másolást generál.
- Minden sor 16 screen byte-ot és 16 color byte-ot másol.
- Nem kell JSR; a kód közvetlenül a makró pozíciójára kerül.
- Normál karakter móddal és multicolor karakter móddal is működik; a Color RAM byte-ok hordozzák minden cella karakterszínét / multicolor engedély bitjét.

Tipikus párosítás Charset Canvas-szal:

```
* = $0801
SEI
LDA #$00
STA $D021
LDA #$05
STA $D022
LDA #$0D
STA $D023
.map_copy16x16 $3000, 12, 4
loop:
JMP loop

.incbin "16x16+ram+color.bin", $3000
.incbin "16x16+charset.bin", $2000
```

SPRITE_ANIM

Minden híváskor lépteti egy sprite animációs frame-jét, és frissíti a VIC-II sprite adat-pointerét. Tárolj frame-enként egy byte-ot egy táblában (a sprite adat-lap száma = `data_address / 64`), mutass rá a SPRITE_ANIM-mal, és hívd game frame-enként egyszer — nem kell JSR.

Mező	Leírás
Sprite #	Sprite szám 0–7
Frame list address	A frame-tábla hex címe — frame-enként egy byte, minden byte = sprite lap (<code>data_addr / 64</code>)
Frame count	Frame-ek száma összesen (1–255)
Frame ZP	Frame-számlálóként használt zero-page byte (pl. <code>FB</code>)

Generált ASM (sprite 0, 4 frame, ZP `$FB` , lista a `$C100` -on):

```

    INC $FB          ; frame-számláló léptetése
    LDA $FB
    CMP #$04         ; frame-szám
    BCC *+6          ; ha számláló < szám, ugorjuk át a resetet
    LDA #$00
    STA $FB
    TAX              ; X = aktuális frame-index
    LDA $C100,X      ; ehhez a frame-hez tartozó sprite adat-lap betöltése
    STA $07F8        ; sprite 0 pointer frissítése ($07F8 + sprite#)
```

Méret: 19 byte.

Tipikus használat:

```

frameTable:
    .byte $21, $22, $23, $24    ; 4 frame a $0840, $0880, $08C0, $0900 címeken

gameLoop:
    WAIT_RASTER ($FF)
    SPRITE_ANIM (sprite=0, list=$C100, count=4, zp=$FB)
    JMP gameLoop
```

Expert-mód szintaxis:

```

.sprite_anim spriteNum, frameListAddr, frameCount, zpByte
; példa:
```

```
.sprite_anim 0, C100, 4, FB
```

Tipp: tedd a frame-táblát RAWBYTES blokként fix címre. A számláló ZP byte-ot (`$FB`) `$00` -ra kell inicializálni az első hívás előtt. Ha a kódod máshoz használja a `$FB` -t, válassz szabad ZP helyet.

SCORE_BCD

Egy fix pontszámot ad hozzá egy memóriában tárolt több-byte-os BCD pontszámhoz, majd minden számjegyet screen RAM-ba renderel screen-code karakterként. A 6502 decimal módot (`SED` / `CLD`) használja carry-biztos BCD aritmetikához — nem kell kézzel carryzni.

Mező	Leírás
Score address	A BCD pontszám-byte-ok hex címe (pl. <code>C200</code>). Alsó byte először.
Digits	BCD byte-ok száma (minden byte két számjegyet tárol: <code>\$99</code> = „99”). 4 byte = akár 99999999.
Add points	Híváskénti decimális hozzáadandó érték (pl. <code>100</code>).
Screen address	Hová írja a számjegy screen code-okat (pl. <code>0400</code>). Számjegyenként egy byte (felső nibble először).

Expert szintaxis:

```
.score_bcd $C200, 4, 100, $0400
```

Generált ASM (4 byte, +100 pont, pontszám a `$C200` -on, screen a `$0400` -on):

```
SED                ; BCD / decimal mód engedélyezése
CLC
LDA $C200          ; 0. byte (1-2. számjegy)
ADC #$00           ; a 100 alsó byte-ja BCD-ben = $00
STA $C200
LDA $C201          ; 1. byte (3-4. számjegy)
ADC #$01           ; a 100 középső byte-ja BCD-ben = $01 (carry propagál)
STA $C201
LDA $C202
ADC #$00
STA $C202
LDA $C203
ADC #$00
STA $C203
CLD                ; vissza bináris módba

; számjegyek renderelése
```

```

    LDX #$00
digit_loop:
    LDA $C200,X
    PHA
    LSR : LSR : LSR : LSR ; felső nibble → alsó nibble
    ORA #$30             ; + '0' screen code
    STA $0406,X          ; jobbra igazítva: screen_addr + (digits*2 - 2) - X*2
    PLA
    AND #$0F             ; alsó nibble
    ORA #$30
    STA $0407,X
    INX
    CPX #$04
    BNE digit_loop

```

Méret: $3 + \text{digits} \times 8$ byte (SED + CLC + CLD overhead + BCD byte-onként 8 byte az ADC-hez + display hurok).

Expert-mód szintaxis:

```
.score_bcd $C200, 4, 100, $0400
```

Tipp: inicializáld a pontszám-byte-okat `$00`-ra induláskor. A pontszám-cím zero page-ben vagy abszolút RAM-ban legyen — ne ROM-ban. A screen-cím a legbaloldali számjegycellára mutasson; a számjegyek balról jobbra íródnak (legnagyobb helyi értékű byte először).

BCD tartomány: $\text{digits}=4$ byte → 8 decimális számjegy → max pontszám 99 999 999. Minden byte két BCD számjegyet kódol: `$00` – `$99`.

10. Debugger integráció

Az app **RetroDebugger**t támogat külső C64 debuggerként. Ez a lefordított programból generált breakpointokat, szimbólumokat és autostart flag-eket kap.

RetroDebugger

A [RetroDebugger](#) egy platformfüggetlen Commodore 64 debugger breakpoint-támogatással, memória-vizsgálattal és címke-tudatos disassemblyvel.

Beállítás: nyisd meg a **Settings** → **Configure RetroDebugger executable** menüt, és mutasd a `RetroDebugger` binárisra.

Indítás: kattints a **Debug (RetroDebugger)** gombra az eszköztáron. Az app:

1. Assembleálja a programot egy `.prg` fájlba egy ideiglenes könyvtárban.

2. Ír egy **breakpoints fájl** (`breakpoints.txt`) — jelölt blokkonként egy `break $ADDR` .
3. Ír egy **symbols fájl** (`symbols.txt`) Vice/RetroDebugger címke-formátumban (`a1 C:addr .name`). Minden LABEL és CONST blokk benne van.
4. C64Debugger-stílusú sidecarokat is ír a lefordított PRG mellé: `.dbg` , `.sym` és `.vs` .
5. Elindítja a RetroDebugget:

```
RetroDebugger -prg <file.prg> -breakpoints <breakpoints.txt> -symbols <symbols.txt> [flags]
```

Breakpoint blokkok

Kattints a breakpoint ikonra (●) bármelyik utasításblokkon, hogy breakpointként kapcsold. A breakpointolt blokkok pirosan emelődnek ki. A címeik minden debugger-indításkor a breakpoints fájlba kerülnek.

Debugger flag-ek (Options fül)

Kapcsoló	Flag	Hatás
<code>-jmp BE</code>	<code>-jmp \$ADDR</code>	Betöltés után közvetlenül a program kezdőcímére ugrik
<code>-unpause BE</code>	<code>-unpause</code>	Betöltéskor azonnal folytatja a debugget
<code>-wait BE</code>	<code>-wait <ms></code>	<code><ms></code> milliszekundumot vár a folytatás előtt — 500 ms vagy 1000 ms

Tipp: a legtöbb programnál kapcsold be a `-jmp` -et és a `-unpause` -t az azonnali autostarthoz. Használd `-wait 500` -at vagy `-wait 1000` -et, ha a programod IRQ-kat vagy SID zenét állít be, aminek idő kell az inicializáláshoz az első raszter előtt.

11. Tudásbázis linkek

Gyorsreferencia linkek az appban a **Knowledge Base** alatt:

Erőforrás	URL
6502 Opcodes Reference	http://www.6502.org/tutorials/6502opcodes.html
C64 KERNAL Functions	https://sta.c64.org/cbm64krnfunc.html
C64 Memory Map	https://sta.c64.org/cbm64mem.html
C64 Color Codes	https://sta.c64.org/cbm64col.html
VIC-II Article	https://www.cebix.net/VIC-Article.txt
C64 Codebase	https://codebase.c64.org/
The Turbo Assembler	https://turbo.style64.org/
RetroDebugger	https://github.com/slajerek/RetroDebugger/

12. D64 export és futtatás

Az 1.5.1 verzió hozzáadja a lehetőséget, hogy a programodat (és további adatfájlokat) egy C64 D64 lemezképbe csomagold, és VICE-ban indítsd — vagy a lemezképet máshol való használatra exportáld.

Osztott Run gomb

Az eszköztár **Run** gombja **osztott gombra** cserélődött:

Rész	Művelet
► Run (fő)	A jelenleg kiválasztott futtatási módot hajtja végre
▼ (nyíl)	Megnyitja a mód-választót

Elérhető futtatási módok:

Mód	Leírás
Run as PRG	Assembleálás egy ideiglenes <code>.prg</code> -be, és VICE közvetlen indítása. Klasszikus viselkedés.
Run via D64	Assembleálás, egy <code>.d64</code> lemezkép építése (c1541-gyel), a beállított extra fájlok hozzáadása, majd VICE indítása a lemezről. Ezt használd, amikor a programod futásidőben tölt fájlokat (pl. a LOADFILE makróval).
Run on hardware	Assembleálás PRG-be, és küldés egy 1541 Ultimate / Ultimate 64 eszközre a helyi hálózaton. Lásd a 13. szakaszt .

A kiválasztott mód munkamenetek közt mentődik.

Export to D64 dialógus

A **Save PRG** ▼ legördülőn → **Export to D64** menüponttal nyílik. A dialógus lehetővé teszi:

1. A **lemeznev** (max 16 karakter) és a **program neve** beállítása — ezek jelennek meg a C64 lemez-könyvtárban.
2. **Extra fájlok hozzáadása** — kattints a + -ra bármely binárisfájl (`.prg` , `.bin` , `.sid` , stb.) kiválasztásához. Minden extrához:
 - **Name** — hogyan jelenik meg a D64 könyvtárban (max 16 karakter, automatikus nagybetű).
 - **Addr** (betöltési cím, opcionális) — ha megadott, egy 2 byte-os PRG fejléc kerül elé. Hagyd üresen, hogy nyers byte-okat írjon fejléc nélkül.
 - **Dst** (kitömörítési cél, csak EXO-val) — hova rakja a depacker az adatot a C64-en. Ha az EXO engedélyezett, az extra `exomizer mem -l <Addr> file,<Dst>` -vel tömörödik a D64-be írás előtt. A +2 safety-offset kompenzáció automatikusan alkalmazódik.
 - **EXO** — jelölőnégyzet, amely bekapcsolja a visszafelé `mem` -módú tömörítést ehhez a bejegyzéshez. A lemezen lévő méret jellemzően az eredeti 5–20%-a.
3. Kattints az **Export**-ra a `.d64` fájl generálásához a VICE `c1541` eszközével.

Párosítás az EXODECRUNCH-csal: amikor EXO=be beállítással szállítasz egy fájlt, az azt olvasó programnak az **Addr** címre kell LOAD-olnia (sec=1, a fájl saját PRG fejléce), majd hívnia az **EXODECRUNCH** makró, hogy visszafelé kicsomagolja a **Dst**-re. A teljes mintáért lásd az `exo-multicolor-demo` mintát.

D64 metaadat a projektekben

A lemeznév, a program neve és az extra fájl-lista a projekt JSON-ban mentődik (a `d64` kulcs alatt). Amikor újratöltesz egy projektet vagy D64 metaadatot tartalmazó mintát, az extrák automatikusan visszaállnak — nem kell minden alkalommal újra hozzáadni őket.

A **loadfile-demo** minta előre be van állítva a `DEMO-COLORS.PRG` -vel extra fájlként. Válaszd ki, nyisd meg a **Run via D64**-et, és kattints a **Run**-ra a teljes betöltési folyamat működés közbeni megtekintéséhez.

Követelmény: a D64 export és a Run via D64 is megköveteli a VICE (`c1541`) beállítását a [Hardver beállításokban](#).

D64 Editor (meglévő lemezkép böngészése és szerkesztése)

A Curve Editor utáni eszköztár-ikon nyitja meg a **D64 Editort** — egy önálló eszközt egy meglévő `.d64` lemezkép közvetlen szerkesztésére, a jelenleg nyitott programtól függetlenül. A fenti Export to D64 dialógustól eltérően (ami mindig egy új lemezt épít a lefordított PRG-ből), a D64 Editor a lemezképet a helyén szerkeszti `c1541` -en keresztül, így egy könnyűsúlyú lemezkezelőként is funkcionál.

Files ▾ menü:

Menüpont	Művelet
New D64...	Célelési út kiválasztása, majd egy frissen formázott, üres lemezkép létrehozása ott.
Open D64...	Meglévő <code>.d64</code> fájl kiválasztása és a directory betöltése.
Save As...	A jelenleg nyitott lemezkép másolása egy új útvonalra, és a szerkesztés folytatása a másolaton.
Run in VICE	A jelenleg nyitott lemezkép közvetlen indítása VICE-ban (<code>-drive8type 1541</code>).

Eszköztár:

Ikon	Művelet
Add program	Helyi fájl kiválasztása és beírása a lemez directory-jába.
Extract selected	A kijelölt bejegyzés byte-jainak mentése helyi <code>.prg</code> fájlba.
Rename selected	A bejegyzés nevének helyben szerkesztése a táblázatban — Enter megerősít, Escape megszakít.
Delete selected	A kijelölt bejegyzés törlése a lemezről.
Refresh	A directory újraolvasása, pl. ha közben másik eszközzel módosítottad a lemezt.

Program hozzáadása: ha már `.prg` -re végződő fájlt választasz, csak egy **Name** és egy lemez **Type** (PRG/SEQ/USR/REL) mezőt kér — a `.prg` már tartalmazza a saját betöltési cím fejlécét, így változtatás nélkül kerül fel. Bármely más fájl (pl. nyers `.bin`) kiválasztásakor emellett megjelenik:

- **Load address** (hex, opcionális) — 2 byte-os PRG fejléc elhelyezése ezen a címen; hagyd üresen, hogy a byte-ok nyersen kerüljenek fel.
- **Decompress address** (hex, opcionális) — csak Exomizerrel együtt használatos; a célcím, ahová a depacker kicsomagolja az adatot.
- **Exomizer** jelölőnégyzet — a fájl tömörítése feltöltés előtt, ugyanazokkal a `mem` / `sfx` tömörítési módokkal, mint a fenti Export to D64 dialógus extra fájljainál.

A directory lista ugyanazzal a fonttal és nagybetűs stílussal jelenik meg, mint egy valódi C64 `LOAD"$",8` lista.

Követelmény: az Export to D64-hez hasonlóan a D64 Editor is megköveteli a VICE (c1541) beállítását a [Hardver beállításokban](#). Minden művelet (hozzáadás/törlés/átnevezés/kimentés) azonnal a lemezen lévő .d64 fájlra hat — nincs külön "mentés" lépés.

12b. CRT export (Magic Desk 64K cartridge)

A **Menu** → **Build** → **Build CRT** egy Commodore 64 cartridge képet (.crt , **cartridge type 19 — Magic Desk / Domark / HES Australia**) állít elő, amely fut VICE-ban, TheC64-en, valódi hardveren EasyFlash / Kung Fu Flash révén, és 1541 Ultimate II+ cartridge slotokban. Elérhető **blokk módban** és **Expert módban** is, és — a jelenlegi buildtől — **UltimateBasic módban** is.

Mi kerül a cartba

- **8 × 8 KB bank** a \$8000 -en, bank-váltás a \$DE00 -n át (Magic Desk konvenció: alsó 3 bit = bank, 7. bit = cart letiltása).
- A **0. bank** egy 128 byte-os fejléctet + boot loadert tartalmaz:
 - \$8000/\$8002 cold + warm start vektorok a \$8009 -re mutatnak.
 - \$8004-\$8008 = a CBM80 szignatúra, amit a KERNAL reset kód megkövetel.
 - \$8009-\$807F = a loader: SEI / stack init / JSR \$FDA3 (IOINIT) / JSR \$FD50 (RAMTAS) / JSR \$FD15 (RESTOR) / JSR \$FF5B (CINT), majd egy byte-másoló hurok, amely a payloadot a cart ROM-ból RAM-ba streameli, és bankot vált, amikor a \$FC eléri az \$A0 -t. A végén egy kicsi **kilépő stubot** másol a \$0100 -ra, letiltja a cartot LDA #\$80 : STA \$DE00 -val, és a belépési pontra JMP -el.
- A **payload** a 0. bankban a \$8080 -on kezdődik, és szükség szerint az 1–7. bankba folyik át. Maximum payload = $8 * 8192 - 128 = 65\,408$ byte .

Betöltési cím és belépési pont

A Build CRT sosem használ Exomizert (a depacker nem futtatható cart ROM-ból). A szabványos autostart pipeline-nal fordítja az aktuális fület, és a betöltési címet a PRG fejlécből, a belépési pontot a SYS célból veszi:

- **Blokk / Expert mód BASIC SYS stubbal be:** load = \$0801 , belépés = a SYS cél (jellemzően \$080D vagy a felhasználó originje).
- **Blokk / Expert mód BASIC SYS stubbal ki:** load = felhasználói origin (a klasszikus \$0801 → \$C000 fallbackkel), belépés = betöltési cím.
- **UltimateBasic mód:** a load és a belépés is a UB fordító map-jéből jön (build.map.loadAddress). A payloadon belüli UB autostart stub ezután pontosan úgy fut le, ahogy lemezzről LOAD "...",8,1 : RUN után tenné.

Az ASM kimenetben látott origin cím megőrződik; a loader egyszerűen bemásolja a lapos memória-képet a PRG-ból RAM-ba, és a belépési pontra ugrik, miután a cart ROM ki van mappelve.

Méretkorlát

Mivel a payload lineárisan tárolódik, és az assembleProgramToPrg() egy lapos minAddr..maxAddr buffert ad vissza nullákkal töltött részekkel, egy egymástól távoli ORG szegmensekkel rendelkező program (pl. \$0801 + \$C000 + \$E000) minden közbeeső byte-ot beleszámol a 65 408 byte-os keretbe. Ha túlléped a korlátot, a build saveCrtTooLarge hibával megszakad — vagy tömörítsd a memória-elrendezést, vagy oszd szét az adatot.

⚠ Kiemelt figyelmeztetés — olvasd el CRT szállítása előtt

A loader a szabványos reset szekvencia részeként hívja a KERNAL **RESTOR (\$FD15)**-öt. Ez szándékosan visszairja a szabványos I/O vektorokat a **\$0314/\$0315** , **\$0316/\$0317** , **\$0318/\$0319** , **\$0328/\$0329** és társai címeken a ROM alapértékeikre.

Következmények:

- **Minden IRQ / NMI / BRK hook, amit a CRT bootolása előtt állítottak be, törlődik.** A programodnak magának kell beállítania őket a belépés után — pontosan úgy, mint egy friss **LOAD "",8,1 : RUN** szalagról/lemezről.
- Az **UltimateBasic programok**, amelyek nem alapértelmezett KERNAL vektorok élő voltára támaszkodnak a belépéskor, explicit **SYS** -t vagy init hívást igényelhetnek az autostart stubban. A szabványos UB autostart out of the box működik; a vektorokat **RUN előtt** hookoló bővítő könyvtárak nem.
- A **CIA1 / CIA2** újrainicializálódik az **IOINIT** által. Az egyéni timer-beállításokat (raszter IRQ-k, zenelejátszó CIA-A) újra kell programozni a belépés után.
- A cart egy 8 byte-os stubból van letiltva a **RAM-ban a \$0100 -on**, hogy a **STA \$DE00** -t ne szakíthassa meg egy hibás ROM fetch. Ne támaszkodj arra, hogy a **\$0100-\$0107** a stack-tető képét tartalmazza belépéskor — az első RAM push felülírja a stubot.

Ha egy CRT fut VICE-ban, de valódi hardveren elbukik, először azt ellenőrizd, hogy a program feltételez-e egy konkrét KERNAL vektort vagy CIA timer állapotot a belépéskor. Állítsd be az állapotot explicit módon az init rutinodban, és mindkettőn ugyanúgy fog viselkedni.

Kompatibilitás

Platform	Állapot
VICE (x64sc , x64)	Működik a File → Attach cartridge image menüvel.
TheC64 / TheC64 Mini	Működik a beépített cartridge loaderrel.
Kung Fu Flash	Működik — natív Magic Desk mód.
EasyFlash cartridge	Működik, Magic Deskként programozva.
1541 Ultimate II+ / Ultimate 64	Működik a Cartridge → Load cart image menüvel.
Chameleon / Turbo Chameleon	Működik.

13. Hardver beállítások

A **Settings** → **Hardware Settings...** menüponttal nyílik az eszköztár menüjéből. Minden külső hardver-útvonal és hálózati konfiguráció itt található.

VICE emulátor

Beállítás	Leírás
Select VICE	Tallózz az x64sc (vagy x64) VICE futtatható fájlra
Status	Mutatja, hogy a futtatható útvonala érvényes és elérhető-e

A VICE szükséges a **Run as PRG**, **Run via D64** és **Export to D64** műveletekhez.

Beállítás	Leírás
Select Exomizer	Tallózz az <code>exomizer</code> futtatható fájlra
Border flash during decompression	Bekapcsolva az SFX-tömörített PRG-k az exomizer beépített <code>-x1</code> gyors keret-villantás effektjét használják; kikapcsolva <code>-n</code> kerül átadásra a csendes kitömörítéshez
Status	Mutatja, hogy a futtatható útvonala érvényes és elérhető-e

Munkafolyamat:

- Telepítsd az Exomizer binárist:
 - Windows:** töltsd le az előre elkészített `win32/exomizer.exe` -t a <https://bitbucket.org/magli143/exomizer/wiki/Home> vagy a <https://csdb.dk/release/?id=244342> címről.
 - macOS:** `brew install exomizer` (Magnus Lind hivatalos 3.1.2 buildjét telepíti).
- Állítsd be az útvonalat a **Hardware Settings** → **Exomizer szakaszban**.
- Kapcsold be a **Run with Exomizer** jelölőnégyzetet a **Settings menüben**.
- Minden **Run** művelet (PRG, D64, hardver) és **Build** művelet (Build PRG, Build D64) mostantól `exomizer sfx sys` -szel tömöríti a lefordított programot indítás vagy mentés előtt.

Az Exomizer Windowson és macOS-en ugyanúgy működik — a CLI-t a Tauri backend hívja; az integrációban semmi nem platformfüggő.

Két tömörítési mód használódik belsőleg:

Mód	Használja	Hívási konvenció
<code>sfx sys</code>	Build/Run with Exomizer kapcsoló (fő PRG út)	Önkicsomagoló PRG beépített dekruncherrel; a Border-flash beállítás vezérli <code>-x1</code> vs <code>-n</code>
<code>mem</code> (visszafelé)	Run via D64 → per-fájl EXO jelölőnégyzet	Minden extra fájlt <code>mem</code> -módú streamre tömörít; a program futásidőben csomagolja ki az EXODECRUNCH makróval és egy előre elkészített depackerrel (<code>samples/exo-decrunch.bin</code>)

Tipp: ha az Exomizer útvonal nincs beállítva, de a jelölőnégyzet be van kapcsolva, indítás helyett egy beszédes hiba-toast jelenik meg. Kapcsold ki a jelölőnégyzetet tömörítés nélküli futtatáshoz.

Integrációs teszt: `cargo test --test exomizer_integration` (a `src-tauri/` -ban) ellenőrzi a teljes mem-módú tömörítés + kitömörítés round-tripet egy 6502 emulátoron.

Retro Debugger

Beállítás	Leírás
Select RetroDebugger	Tallózz a <code>RetroDebugger</code> binárisra
Status	Mutatja, hogy az útvonal érvényes-e

A teljes debugger dokumentációért lásd a [10. szakaszt](#).

C64 Ultimate / 1541 Ultimate

Lefordított PRG-ket futtass közvetlenül valódi hardveren a helyi hálózaton az Ultimate REST API-n keresztül.

Beállítás	Leírás
Host (IP)	Az eszköz IP-címe (pl. 192.168.1.100)
Password	Opcionális — ha az eszköz hitelesítést igényel
Test connection	Teszt-kérést küld a /v3/runners/info -ra; OK-t vagy hibát mutat

Munkafolyamat:

- 1. Csatlakoztasd az 1541 Ultimate / Ultimate 64-et a helyi hálózathoz.
- 2. Add meg az IP-címét (és jelszavát, ha van) a Hardware Settings-ben.
- 3. Válaszd a **Run on hardware**-t az osztott run menüből.
- 4. Kattints a ► **Run**-ra — a PRG lefordul, és HTTP POST-tal a /v3/runners/prg -re kerül. Az eszköz azonnal betölti és futtatja a C64-en.

Tipp: nem kell USB kábel vagy driver — a REST API be van építve az Ultimate firmware-be. A számítógépnek és az eszköznek ugyanazon a helyi hálózaton kell lennie.

14. Vizuális szerkesztők (Toolkit)

Az eszköztár **Toolkit** menüje csoportosítja a vizuális adat-szerkesztőket, amelyek mind egy közös Files menüt (**Files** ▾) osztanak meg: Load BIN / Save BIN / Export to blocks / Save to D64. Minden szerkesztő nyers **.bin** adatot állít elő, amely **INCBIN** -nel programba helyezhető, vagy közvetlenül D64 lemezre adható a **Save to D64** menüponttal.

A vizuális szerkesztő dialógusok a fejlécüknél húzva mozgathatók a teljes Visual Assembler munkaterületen. Egy mentett pozíció nélküli dialógus közepén nyílik; mozgatus után az utolsó pozíciója a UI beállításokban tárolódik, és a következő megnyitáskor visszaáll.

Hi-Res / Multicolor Editor

Pixel-szintű bitmap szerkesztő 320×200 hi-res és 160×200 multicolor módokkal. Toolkit → Hi-Res Editor menüvel nyílik.

Funkció	Leírás
Mode toggle	A Multicolor jelölőnégyzet vált hi-res (cellaként mono) és multicolor (cellaként 4 szín) közt.
Eszközök	Ceruza, radír, vonal, téglalap, kitöltött téglalap, ovális, kitöltött ovális, flood fill.
Spray eszköz	Airbrush-stílusú festő, amely pixeleket szór a kurzor köré rajzolás közben.
Spray intenzitás	A spray eszköz melletti legördülő vezérli, mennyire sűrű minden spray húzás.
Színpaletta	Előtér (ink) + papír (háttér) választók. A multicolor mód automatikusan követ 3 cellakénti extrát.
Undo / Redo	Húzásonkénti előzmény, ctrl-Z / ctrl-Y.
Grid + Raster	Opcionális 8×8 rács és raszter-sor overlay cella-igazításhoz.
Kép importja	A canvas-ra dobott PNG/JPEG/GIF automatikusan a 16 színes C64 palettára kvantálódik.
Export blokkok	BYTE/RAWBYTES blokkokat fűz a programhoz a kódolt bitmap, screen és color adattal.

Funkció	Leírás
Export <code>.bin</code>	A natív multicolor formátumot menti (10000 byte: 8000 bitmap + 1000 screen + 1000 color), készen a \$2000-re való LOADFILE-hoz.

Sprite Editor

24×21 pixeles sprite szerkesztő több-frame-es animációval. Toolkit → Sprite Editor menüvel nyílik.

Funkció	Leírás
Frame-ek	Frame-ek hozzáadása / eltávolítása / átrendezése; a frame-csík alul látszik.
Mode	Mono / multicolor kapcsoló.
Eszközök	Ceruza, kitöltés, vonal, téglalap, kör — plusz vízszintes tükrözés, függőleges tükrözés, balra/jobbra/fel/le tolás (opcionális körbefordulással). A shape eszközök élő előnézetet mutatnak húzás közben; elengedésre véglegesít.
Undo / redo	Teljes undo/redo verem frame-enként. Ctrl/Cmd+Z / Ctrl/Cmd+Y vagy az eszköztár gombjai.
Kép importja	PNG vagy JPEG importja a Files → Import image menüvel. Az importáló minden pixelt a legközelebbi C64 palettaszínre képez, és az aktuális frame-be írja.
Animáció előnézet	Play / Stop konfigurálható sebességgel.
Export blokkok	Egy RAWBYTES blokkot szúr be 64 byte-ra igazított címre minden frame-hez, plusz egy sprite pointer beállítást.
Save <code>.bin</code>	Frame-enként 64 byte-ot ír (nyers sprite adat kitöltés nélkül).

C64 karakter-ROM böngésző („Char map”)

A C64 karakter-ROM (a beépített PETSCII font) csak olvasható nézője. A Toolkit menü **C64 chargen** menüpontjával nyílik. Hasznos egy glif screen code-jának megtalálásához, mielőtt `RAWBYTES`-szal vagy `TEXT`-tel kiírnád.

Funkció	Leírás
Két karakterkészlet	1. fül: Set 1 — Upper/Graphics (bekapcsolás utáni alap). 2. fül: Set 2 — Lower/Upper (a \$0E váltás után).
Glif-rács	16×16-os rács mind a 256 karakterrel. Kattints egy glifre a részlet-paneljéhez: nagyított 8×8 pixel nézet, screen code (decimális + hex), PETSCII kódok (alap és shiftelt is), és a nyers 8 byte-os bitmap.
Részlet-panel	Mutatja a kiválasztott glif screen code-ját, PETSCII kódjait és a nyolc nyers byte-ot — készen a <code>RAWBYTES</code> vagy <code>BYTE</code> blokkba illesztéshez.
Csak olvasható	Itt nincs szerkesztés — a glifek módosításához használd a Character Editort (lentebb).

Character Editor (Charset)

256 karakteres 8×8 charset szerkesztő. Toolkit → Character Editor menüvel nyílik.

Funkció	Leírás
Load ROM	A C64 ROM charsetet közvetlenül a VICE <code>chargen</code> -jéből importálja (nincs fájlválasztó).
Load <code>.bin</code>	Külső 2048 byte-os charset binárist importál.

Funkció	Leírás
Karakter-előnézet	16 széles rács mind a 256 glif-fel, az aktuális cella kiemelve.
Pixel szerkesztő	8×8 egy-karakteres szerkesztő toggle/invert/clear eszközökkel.
Karakterenkénti szín	Alapértelmezett Color RAM értéket tárol minden glifhez. Multicolor karakter módban ez a cellákénti multicolor engedély bitet és a karakter saját alsó-3-bites színét is megőrzi.
Metaadat round-trip	Charset Canvas / Map munkafolyamatokból kompatibilis adat betöltésekor a karakterenkénti szín-metaadat megőrződik, így a szerkesztések folytathatók a szín-szándék elvesztése nélkül.
Export blokkok	RAWBYTES-t fűz a \$0800-ra (2. blokk) vagy a \$3800-ra (7. blokk) a kódolt charsettel.

Charset Canvas Editor

Teljes-canvas charset festő képernyők építéséhez egy teljes 256 karakteres charsetből. Toolkit → Charset Canvas menüvel nyílik.

A canvas 16×16 karakter. Mono módban ez egy 128×128 pixeles munkaterületet ad; multicolor karakter módban egy 64×128 széles-pixeles munkaterületet a C64 valódi karakter-multicolor szabályaival.

Funkció	Leírás
Mono / multicolor	A mono mód 1 bites 8×8 glifeket tárol. A multicolor mód kétbites vízszintes pixel-párokat tárol, és a használt cellákat Color RAM 3. bittel jelöli.
C64 színmodell	A háttér a \$D021 -et használja; a megosztott multicolor 1 a \$D022 -t; a megosztott multicolor 2 a \$D023 -at; minden karakter saját színe a Color RAM 0–2. bitjéből jön.
Rajzeszközök	Ceruza, radír, vonal, téglalap, ovális és flood fill karakterhatárokon át működik.
Spray eszköz	Airbrush-stílusú rajzolás, amely pixeleket szór a szomszédos cellák/karakterek közt.
Spray intenzitás	A spray eszköz melletti legördülő beállítja a húzás sűrűségét.
Grid kapcsoló	A Grid jelölőnégyzet megjeleníti vagy elrejt a 16×16 karakter-rácsot.
Save charset .bin	A 2048 byte-os karakter-bitmap adatot menti.
Save 16×16 map + Color RAM .bin	256 screen code-ot, majd 256 Color RAM értéket ment. Ezt a MAP_COPY16X16 -tal használd.
Betöltés	Betölthet charset-canvas mentéseket, sima charset adatot és kompatibilis Character Editor charset adatot, beleértve a tárolt karakterenkénti színeket, ha jelen vannak.

Fontos C64 korlát: multicolor karakter módban a két megosztott szín globális az egész képernyőre (\$D022 / \$D023). Csak a karakter saját színe cellánkénti, és az a 0–7 színekre korlátozódik, mert a Color RAM 3. bit választja a multicolor módot.

Map Editor (több-rétegű tilemap-ek)

Réteges tilemap szerkesztő statikus díszlethez, sprite spawn map-ekhez, ütközési adatahoz és hasonlókhöz. Toolkit → Map Editor menüvel nyílik.

Funkció	Leírás
Rétegek	Több nevesített réteg, mindegyik saját tileset-tel és átlátszatlansággal.

Funkció	Leírás
Ecsetek	Egy-tile, kitöltés, vonal, téglalap és kör módok. A shape eszközök élő előnézetet mutatnak húzás közben; elengedésre véglegesít.
Undo / redo	Teljes undo/redo verem rétegenként. Ctrl/Cmd+Z / Ctrl/Cmd+Y vagy az eszköztár gombjai.
Clear menü	Rétegenkénti vagy teljes-map törlés megerősítéssel.
Kép importja	Dobj be egy tilemap PNG-t; a szerkesztő automatikusan tile-okra szeleteli.
Copy / paste	Másolj egy kijelölt tile-tartományt, majd illeszd be normálisan, vagy használd átlátszó beillesztést, hogy az üres tile-ok átlátszók maradjanak.
Multicolor-tudatos tile-színezés	Kompatibilis charset metaadattal a festés a tile tárolt Color RAM alapértékeit használja (a multicolor-hoz kapcsolódó kódolással) egy általános lapos szín helyett.
Egyéni charset színek	Ha egy charset <code>charColors</code> metaadatot hordoz, a Map Editor a kiválasztott tile alapértelmezett Color RAM értékét használja festés közben.
Export blokkok	RAWBYTES blokkokat bocsát ki a tileset grafikához + map adathoz.
Save Screen RAM (.bin)...	Csak az aktuális map réteg screen code-jait menti (40×25 = 1000 byte).
Save Screen RAM + Color RAM (.bin)...	Screen code-okat a Color RAM értékekkel összefűzve ment egyetlen 2000 byte-os fájlba (<code>screen[0..999]</code> , majd <code>color[0..999]</code>). Ezt a MAP_COPY makróval használd (Combined .bin mód) mindkettő egy művelettel való visszaállításához futásidőben.

SID Editor (3-voice tracker)

Több-hangszeres 3-voice tracker Web Audio előnézet-motorral. Toolkit → SID Editor menüvel nyílik.

Hangszerenkénti vezérlők:

- Hullámforma jelölőnégyzetek (TRI / SAW / PUL / NOI) — több hullámforma OR-olható.
- ADSR (attack / decay / sustain / release) drag-grafikonként a négy csúszka fölött.
- Pulzusszélesség csúszka (0-4095) opcionális ring/sync flag-ekkel.
- Filter routing jelölőnégyzet voice-onként; globális filter cutoff / resonance / volume / mód (LP/BP/HP).

Tracker rács:

- 3 voice × akár 7 pattern × 32 sor = 7 × 32 = max 224 sor (a 8 bites sor-számláló korlátozza).
- Soronként: note + hangszer-index. Az üres sorok az előző note-ot tartják.
- Válassz egy cellát normálisan, vagy tartsd a **Shift**-et kattintás vagy nyílbillentyű közben egy téglalap-kijelölés kiterjesztéséhez sorokon és a három voice bármelyikén át. A kijelölt területen belüli jobbklikk megőrzi a tartományt.
- A Copy, Cut, Paste és Clear elérhető az ikon-eszköztárból és az ikon-alapú context menüből. A `Ctrl/Cmd+C` és `Ctrl/Cmd+V` ugyanazon a téglalap-kijelölésen működik.
- Harmónia segéd: válassz alaphangot, akkord-típust és oktávot, hallgasd meg az akkordot az aktuális hangszerrel, majd szúrd be a hangzatot közvetlenül a trackerbe. Az elérhető típusok: Major, Minor, Diminished, Augmented, Sus2, Sus4, Dominant 7, Major 7, Minor 7, 6, Minor 6, 9, b9, #9, Dim7, 7sus4.
- Arpeggio segéd: hallgasd meg vagy szúrd be a kiválasztott akkordból 4-, 8- vagy 16 lépéses note-futamokat fel, le vagy fel/le irányban.
- A **Preview row** meghallgatja a kiválasztott sort mind a három voice-on a pattern-lejátszás indítása nélkül.

- A cella-tartomány beillesztés mostantól a kijelölt tartomány kezdőcellájánál kezdődik, és tisztán megáll a voice- és sor-határoknál, ahelyett hogy a következő oszlopba vagy sorba fordulna.
- A Speed csúszka beállítja az IRQ tick osztót (frame-ek a sorok közt).

Lejátszás és virtuális billentyűzet:

- A Play eszköztár-gomb lejátszás közben Pause-ra, szüneteltetve Resume-ra vált; a Stop leállítja a lejátszást és visszaállítja az állapotot.
- A billentyűzet eszköztár-gomb egy nem-modális zongorát nyit, amely használható marad, míg a SID szerkesztő aktív. Húzd a fejlécét a fő alkalmazás fölé bárhova.
- Engedélyezd az **Insert into tracker**-t, hogy minden lejátszott note az aktuális tracker-kurzorra íródjon, és a következő sorra lépjen. Kapcsold ki a note-ok meghallgatásához szerkesztés nélkül.
- Az akkord- és arpeggio-előnézetek megvilágítják a megfelelő zongorabillentyűket, amikor a billentyűzet nyitva van.

Files menü exportok:

Export	Mit csinál
Save .bin...	A szerkesztő natív szerializált formátumát írja (hangszerek + patternek + sequence).
Export blocks (data only)	Hangszer-táblát + pattern blokkokat fűz a programhoz a * = \$C000 -en.
Export blocks + miniplayer	Hozzáadja a teljes lejátszót (sid_init / sid_irq / sid_play_row / sid_set_voice) plusz a PAL frekvencia-táblákat. Export után tegyél egy JSR sid_init -et a fő kódodba, ahol a zenének indulnia kell.
Export asm (clipboard)	A teljes assembly forrást vágólapra másolja.

Lejátszó ZP használat: \$FB (tick számláló), \$FC (sor-index), \$FD (set_voice temp). Ezek ütköznek, ha a fő kódod használja őket — szükség esetén relokálj Expert módon.

Ismert korlátok:

- Egyetlen lineáris pattern lista (még nincs per-voice sequence tábla).
- A 8 bites sor-számláló 7 pattern × 32 sorra korlátoz.
- A C64 \$D418 globális hangerő megosztott a voice-ok közt — a hangszerenkénti hangerő-csúszka informatív; a sustain szint (az ADSR S -e) a tényleges per-voice hangerő.
- A Web Audio előnézet közelítő: a PWM moduláció, a ring/sync és a SID filter karaktere eltér a valódi chiptől.

Curve Editor

Kész .byte lookup táblákat generál matematikai görbékből — szinusz, easing-ek, háromszög/fűrészfog/négyszög és bounce. Ideális sprite-mozgáshoz, raszter-effektekhez, szín-ciklushoz, vagy bármely előre számolt táblával vezérelt animációhoz. A felső eszköztár **Curve Editor** ikonjával nyílik (a SID Editor gomb mellett).

Görbék:

Sine, Cosine, Linear, Ease In/Out/InOut (Quad és Cubic), Ease In/Out (Circ), Triangle, Sawtooth, Square, és Ease In/Out/InOut Bounce.

Vezérlők:

Vezérlő	Cél
Start / End value	Kimeneti tartomány. 0..255 8 bites módban, 0..320 16 bites módban.
Number of values	Tábla hossza, 4–512 bejegyzés.
Cycles	Hány oszcilláció a táblán át (csak sine/cosine/triangle/sawtooth/square). Törtéket is elfogad (pl. 3.625).
Phase	Fázis-eltolás fokban (csak sine/cosine).
Combine second curve	Egy második görbe keverése Mix / Add / Multiply / Min / Max / Subtract -tel, saját cycles/phase-szel és egy mix-mennyiséggel. Mindkét forrásgörbe szaggatott vezetőként rajzolódik a grafikonra.
Label	Tábla-címke (a görbe nevéből automatikusan javasolva).
Number format	\$XX hex vagy decimális.
Values per line	8 / 16 / 32 byte .byte soronként.

Kimeneti módok:

Mód	Kibocsát
8-bit	Egyetlen .byte tábla (0.255 értékek). LDX #index / LDA table,X -szel olvasható. Opcionálisan kibocsát egy sprite-Y olvasó rutint (<label>_set_y) — LDA <label>,X / STA \$D001+2N — egy választható 0-7 sprite-számhoz.
16-bit	Két párhuzamos byte-tábla — <label>_lo (alsó 8 bit) és <label>_hi (9. bit, 0/1) — ugyanazzal az X-szel indexelve (bejegyzésként 2 byte). A teljes képernyőn átívelő sprite X-hez kell (0.320 > egy byte). Opcionálisan kibocsát egy sprite-X olvasó rutint (<label>_set_x), amely az alsó byte-ot a \$D000+2N -be írja, és beállítja/törli a sprite MSB-jét a \$D010 -ben, egy választható 0-7 sprite-számhoz.

Minden Copy / Insert kimenet egy fejléc-megjegyzéssel kezdődik, amely dokumentálja a görbét, a tényleges min/max tartományt, a bejegyzés-számot és a pontos használatot (melyik regisztert táplálja az egyes táblák).

Előnézet:

- **Grafikon** — a görbe kirajzolva a 0 értékkel **felül** és a maximummal **alul**, a C64 sprite-Y / raszter konvencióhoz igazodva (így amit látsz, azt vezérli a tábla hardveren). A grafikon alatti meta-sor mutatja a byte-számot, a generált értékek tényleges Min / Max-át és a görbe nevé(è)it.
- **Pattogó labda** — animál egy jelölőt a táblán át a **Tempo**-nál (5–240 érték/mp). Tempo 50-nél ez PAL-on (50 Hz) frame-enként egy érték, azaz indexenként egy .wait_raster lépés. Play/Pause és Restart gombok, az utolsó ~24 pozíció halványuló nyoma, és élő Index • Value kijelzés.

Copy / Insert: az eszköztár két ikonja — a **Copy** vágólapra teszi a táblát; az **Insert into editor** blokkokként fűzi a táblát (és az olvasót, ha engedélyezett) az aktuális programhoz. Az újra-beszúrás **lecseréli** a korábbi Curve Editor beszúrást ahelyett, hogy duplikátumokat halmozna (Blokk és Expert módban is működik).

Files menü:

Művelet	Mit csinál
Save curve (.bin)...	A nyers tábla-byte-okat pontosan úgy menti, ahogy a C64 INCBIN -nel olvasná. 16-bit: N lo byte, majd N hi byte.

Művelet	Mit csinál
Load curve (.bin)...	Nyers tábla-byte-okat tölt vissza a szerkesztőbe, az aktuális bitmélység szerint értelmezve (16-bit: első fele lo, második fele hi). A betöltött tábla változatlanul jelenik meg, amíg bármely görbe-vezérlő friss görbét nem generál.
Export demo to blocks	Egy teljes, futtatható sprite demót fűz hozzá: sprite init, raszter-szinkronizált fő hurok, a beágyazott tábla és a labda sprite adata. Az X 0..320-at söpör 8.8 fixpontban a <code>\$D010</code> MSB-vel, míg a tábla vezérli a sprite Y-t — pontosan a szerkesztő előnézetéhez illeszkedve. Az újra-export lecseréli a korábbi Curve Editor beszúrást.

Az előnézet reprodukálása a C64-en: az előnézet **lineárisan, 0 → N-1 → 0 hurokban, frame-enként egy értéket** olvas a táblából. Ennek pontos reprodukálásához vezérelj a táblát ugyanígy (növekd az indexet frame-enként egyszer, fordulj körbe a tábla hosszánál). Egy ping-pong vagy részleges tartományú lejátsszás máshogy mozog, még ha a byte-értékek azonosak is. Egy működő 16 bites sprite-X példáért lásd a `samples/curve-new-demo.asm` -et.